“Design and Development of a Process Debugger Using Linux System Calls”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

SHETTENI TEJA	**2520030419**
A CHARANTEJA	**2520030418**
ASHWITH REDDY	**2520030475**

Under the Esteemed Guidance of

Soumya
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOL



Declaration

The Project Report entitled **"Design and Development of a Process Debugger Using Linux System Calls"** is a record of Bonafide work of **SHETTENI TEJA - 2520030419, A CHARANTEJA - 2520030418, ASHWITH REDDY - 2520030475** submitted in
partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

SHETTENI TEJA	**2520030419**
A CHARANTEJA	**2520030418**
ASHWITH REDDY	**2520030475**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled **"Design and Development of a Process Debugger Using Linux System Calls"** is a bonafide work done and submitted by **SHETTENI TEJA - 2520030419, A CHARANTEJA - 2520030418, ASHWITH REDDY - 2520030475,**

in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Soumya, and my guide Soumya**,** Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the ***Department for Computer Science Engineering*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

SHETTENI TEJA 2520030419

A CHARANTEJA 2520030418

ASHWITH REDDY 2520030475

Abstract

Debugging is one of the most time-consuming activities in software development, as programmers often struggle to understand why a running program crashes, hangs, or produces incorrect output. General-purpose debuggers are powerful but complex, and they hide the operating system mechanisms that make debugging possible. To tackle this, our project focuses on the Design and Development of a Process Debugger Using Linux System Calls. Our core aim was to build a lightweight, command-line debugger in C that lets a user control and inspect a running program, while demonstrating how the operating system supports process control.

The debugger is structured around four interconnected modules. The Process Control module launches the target program using fork() and exec() and supervises it through the ptrace() system call, with support for run, continue, and single-step execution. The Breakpoint Manager inserts and removes software breakpoints by temporarily replacing an instruction with the INT 3 opcode and restoring the original instruction on resume. The Inspection module reads and writes the registers and memory of the traced process and displays the call stack, while the Signal and Error Handling module uses waitpid() to catch signals such as SIGSEGV and SIGFPE and reports the point of failure.

The most useful feature is the integrated crash analysis, which identifies the faulting instruction and shows the state of registers and memory at the moment of failure, helping users locate bugs faster. The system is developed in C and compiled with GCC on Ubuntu, using Linux system calls, process management, and signal handling concepts. In summary, this project delivers a simple, reliable, and educational debugging tool that makes program errors easier to find and gives practical insight into how an operating system manages, controls, and observes running processes.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		8–19
		1.1 Background of the Project	8–9
		1.2 Problem Statement	10–11
		1.3 Scope of the Project	12–13
		1.4 Objectives of the Project	14–15
		1.5 Importance of the Application	16–17
		1.6 Target Users / Audience	18–19
2	OS Concepts and Background Study		20–27
		2.1 Operating System Concepts Related to the Project	20–21
		2.2 Existing Methods / Algorithms	22–23
		2.3 Proposed Approach	24–25
		2.4 Advantages of the Proposed Approach	26–27
3	System Requirements		28–35
		3.1 Hardware Requirements	28–29
		3.2 Software Requirements	30–31
		3.3 Operating System Environment – Ubuntu / WSL	32–33
		3.4 Development Tools and Compilers	34–35
4	Technology and Tools Used		36–48
		4.1 C / C++ Programming	36–38
		4.2 GCC Compiler	39–40
		4.3 Linux System Calls	41–42
		4.4 POSIX Threads / Synchronization	43–44
		4.5 Linux Commands and Utilities	45–46
		4.6 Other Tools / Libraries Used	47–48
5	System Architecture and Design		49–58
		5.1 Overall System Architecture	49–50
		5.2System Workflow	51–52

		5.3 Flowchart / Data Flow	**53–54**
		5.4 Module Description	**55–56**
		5.5 Input and Output Design	**57–58**
6	**Implementation**		**59–75**
		6.1 Module-wise Implementation	**59–60**
		6.2 Algorithm / Pseudocode	**61–62**
		6.3 Core C / C++ Implementation	**63–65**
		6.4 Linux System Calls / APIs Used	**66–67**
		6.5 Process / Thread Implementation	**68–69**
		6.6 IPC / Synchronization Implementation	**70–71**
		6.7 Memory Management Implementation	**72–73**
		6.8 Optional User Interface Implementation	**74–75**
7	**Execution and Demonstration**		**76–86**
		7.1 Compilation in Ubuntu	**76–77**
		7.2 Execution Commands	**78–79**
		7.3 Sample Inputs and Outputs	**80–82**
		7.4 Screenshots of Execution	**83–84**
		7.5 Demonstration of OS Concepts	**85–86**
8	**Conclusion**		**87–94**
		8.1 Summary of the Project	**87–88**
		8.2 Objectives Achieved	**89–90**
		8.3 Results Achieved	**91–92**
		8.4 Skills and Learning Outcomes	**93–94**
9	**References**		**95–102**
		9.1 Books	**95–96**
		9.2 Research Papers	**97–98**
		9.3 Linux / GNU Documentation	**99–100**
		9.4 Online Resources	**101–102**

CHAPTER 1 INTRODUCTION

1.1 Background of the Project

Every non-trivial program contains bugs. A bug may make a program crash, hang, or quietly produce a wrong answer, and finding its cause is often the slowest part of software development. Programmers therefore rely on a **debugger**: a tool that can start a program under its control, stop it at a chosen place, show what is inside it, and let it run again one step at a time.

Early programmers debugged by printing values and re-running their programs. Modern debuggers such as GDB and LLDB are far more powerful, but they are also large programs with hundreds of commands and many thousands of lines of code. A student who types `break main` and `run` rarely sees what the operating system is doing underneath, and so the debugger remains a black box.

The operating-system support behind a debugger

A debugger cannot do its job alone, because one ordinary process is not allowed to read or change the memory and registers of another. The operating system must grant that permission. On Linux the permission is given through a single system call, `ptrace()` (process trace). A traced process stops whenever it receives a signal; the debugger is told about the stop, and it can then read and write the target's CPU registers and memory and choose how the target resumes. Combined with `fork()`, `execl()`, `waitpid()` and signals, `ptrace()` provides everything needed to build a working debugger.

The Mini Linux Debugger

This project, the **Mini Linux Debugger**, is a small debugger written from scratch in the C language for 64-bit Linux (x86-64). It starts a target program, lets the user set a software breakpoint, continue or single-step the program, view all general-purpose registers, read a word of the program's memory, and trace the system calls that the program makes. The same debugging engine is used by two front ends: a command-line program called `my_debugger` and a graphical program called `debugger_gui` built with GTK+ 3.

File	Lines	Role in the project
`include/debugger.h`	57	Public interface: the `Debugger` structure and the function prototypes
`src/debugger.c`	1085	Debugger core: launch, breakpoint, continue, step, registers, memory, system-call trace, stop
`src/main.c`	213	Command-line front end with the "(debugger)" prompt
`gui/gui.c`	785	GTK+ 3 graphical front end
`test/test_program.c`	20	Sample target program that is debugged in all demonstrations

File	Lines	Role in the project
`Makefile`	—	Builds the three executables: `test program`, `my debugger`, `debugger_gui`

The project is deliberately small. A reader can study the complete source in a few hours and can trace every feature of the debugger to a specific Linux system call. That visibility is the main reason the project was built.

Technology snapshot

Item	Detail
Language	C (compiled with GCC, flags `-Wall -Wextra -g -O0`)
Platform	Linux, x86-64 (developed and tested on Ubuntu)
Key system calls	`fork`, `execl`, `ptrace`, `waitpid`, `kill`
User interfaces	Command line (`my debugger`) and GTK+ 3 window (`debugger gui`)
Target program	`test program` – adds two numbers in `calculate()` and prints `Result = 60`
Build result	The debugger and CLI compile with no warnings under `-Wall -Wextra`

1.2 Problem Statement

Students of operating systems and systems programming learn the theory of processes, signals, system calls and virtual memory, but they seldom see these ideas working together in one practical tool. The debugger is the best such tool, yet the ones in everyday use hide the mechanism completely.

Difficulties faced by learners

1. **The mechanism is hidden.** GDB shows that the program stopped at a breakpoint, but not how. Learners do not see that a single byte of the program's code was replaced by `0xCC` and later restored.
2. **The tools are too large to study.** The source code of a production debugger runs to hundreds of thousands of lines, so it cannot be read as a learning exercise.
3. **The command set is overwhelming.** Beginners must learn many commands before they can do a simple experiment such as "stop here and show me the registers".
4. **Theory and practice are disconnected.** Concepts such as SIGTRAP, `waitpid()` status codes and the saved CPU context stay abstract when there is no small program that uses them.
5. **No simple graphical entry point.** Command-line debuggers are intimidating for first-year students, and a minimal window with a few buttons makes the first experiments much easier.

Gap analysis

What learners need	What common tools provide	Gap
See each system call a debugger uses	Hidden inside the tool	No visibility
A small, readable code base	Very large projects	Cannot be studied
A few simple commands	Hundreds of commands	Steep learning curve
Simple GUI and CLI over the same engine	Separate, complex front ends	Not beginner friendly
Clear messages about what happened	Dense output intended for experts	Hard to interpret

Problem definition

The problem addressed by this project is therefore: **to design and implement a small, readable debugger for Linux that exposes the operating-system mechanisms behind debugging – process creation, tracing, signals, breakpoints, register and memory access, and system-call tracing – and that can be driven from both a simple command line and a graphical**

interface.

Success criteria

Criterion	How it is checked
The target program starts under debugger control	A PID is displayed and the target is stopped before running
A breakpoint stops the program at the chosen address	"Breakpoint hit at 0x401136." is displayed
The program still runs correctly after a breakpoint	The target prints `Result = 60` and exits with status 0
Registers and memory can be inspected	Register table and memory word are displayed
System calls can be traced	Paired ENTER / EXIT lines are displayed
Wrong usage gives a clear message	Messages such as "No running program." appear instead of a crash

1.3 Scope of the Project

The scope describes what the Mini Linux Debugger does and, equally important, what it does not do. The scope was kept narrow on purpose so that every feature could be implemented completely and explained clearly.

Features within the scope

Feature	What it does	Limit
Launch	Starts a program with `fork()` + `execl()` under `PTRACE_TRACEME` and stops it at the start	Program path only; no command-line arguments for the target
Breakpoint	Writes INT3 (`0xCC`) at a hexadecimal address and restores it safely	One breakpoint at a time
Continue	Resumes the program and reports why it stopped	—
Single step	Executes exactly one machine instruction	Shows no source line
Registers	Displays RAX–R15, RIP and EFLAGS in hexadecimal	x86-64 only
Memory read	Reads one 8-byte word at a given address	One word per request
System-call trace	Shows number, name and return value of the next 10 calls	24 names known; others shown as "unknown"
Stop	Terminates the target with SIGKILL	—
Interfaces	Command-line program and GTK+ 3 window using the same core	Single-threaded

Features outside the scope

Not included	Reason
Attaching to an already running process	Keeps the process model simple: the debugger always creates its target
Source-level debugging (line numbers, variable names)	Needs symbol and DWARF parsing, which is a separate large topic
Multiple breakpoints, conditional breakpoints, watchpoints	Single breakpoint is enough to demonstrate the mechanism
Multi-threaded or multi-process targets	Would require `PTRACE_O_TRACECLONE` and per-thread bookkeeping
Non-x86-64 processors	Register names and the INT3 opcode are architecture specific
Disassembly of instructions	Addresses are found with `objdump`; the debugger shows raw words

Assumptions and constraints

- The target is a 64-bit Linux executable built **without PIE** (`-no-pie`), so its code addresses are the same in every run and can be typed in by hand.
- The user enters addresses in hexadecimal. The CLI accepts `0x…` notation; the GUI

accepts digits such as `401136`.

- Debug messages are limited to 4096 bytes (8192 bytes for the system-call trace) because they are written into fixed-size buffers.
- The debugger and the target run on the same machine and under the same user account.

1.4 Objectives of the Project

The overall objective is to build a working debugger that makes the operating-system mechanisms behind debugging visible. This general aim is divided into the specific objectives below. Each objective is tied to a feature of the program and to a way of checking it.

Specific objectives

1. **Control another process.** Learn and demonstrate how `ptrace()` lets a debugger control a target. The child asks to be traced with `PTRACE_TRACEME` and the parent controls it afterwards.
2. **Launch a program under control.** Implement `debugger_launch()` using `fork()` and `execl()` so that the target is stopped at its first instruction and nothing runs before the user decides.
3. **Implement a breakpoint.** Insert the one-byte INT3 instruction (`0xCC`) at an address, detect the trap, and restore the original instruction so the program behaves normally afterwards.
4. **Control execution.** Implement continue and single-step, including the extra work needed to step over an active breakpoint.
5. **Inspect the program.** Read all general-purpose registers with `PTRACE_GETREGS` and read memory with `PTRACE_PEEKDATA`.
6. **Trace system calls.** Use `PTRACE_SYSCALL` together with `PTRACE_O_TRACESYSGOOD` to identify the entry and exit of each call.
7. **Build two interfaces over one core.** Keep the debugger engine free of user-interface code so a CLI and a GTK+ GUI can share it.

Objectives and how they are verified

Objective	Verification in this report
Control another process	PID and "stopped and ready" message (Section 7.3, Session 1)
Launch under control	Target stops before running; `Result = 60` printed only after CONTINUE
Breakpoint	RIP = 0x401137 and memory word ending in `cc` (Section 7.3)
Continue and step	RIP moves 0x401137 → 0x40113a → 0x40113b → 0x40113e on successive steps
Inspect program	Register table and memory word displayed
Trace system calls	Ten ENTER / EXIT pairs displayed (Session 2)
Two interfaces	`my debugger` and `debugger gui` both built from `debugger.c`

Design objectives

Besides the functional objectives above, four design objectives guided every decision in the project. They explain why the code looks the way it does.

Design objective	Meaning	How it was met
Simplicity	A reader should understand any function after reading it once	One responsibility per function; no clever tricks; plain C
Safety	The debugger must never crash or corrupt memory because of wrong input	State checks first, every return value tested, bounded string output
Reusability	The same engine should serve any interface	Core has no input or output code; messages go into a caller-supplied buffer
Visibility	Every operating-system mechanism must be visible in the code	Direct `ptrace`, `fork`, `waitpid` and `kill` calls; no wrapper library

Learning objectives

The project also had learning objectives for the team members. On completing it, each member should be able to:

- explain how a debugger starts a program and keeps control of it;
- explain why a software breakpoint is a patched instruction and why RIP is one byte past it;
- describe what the kernel does at a system-call boundary and how it can be observed;
- read and write another process's registers and memory through the operating system;
- structure a systems program so that its core can be reused by several interfaces.

Objectives and chapters

Objective	Designed in	Implemented in	Demonstrated in
Control another process	5.1, 5.2	6.1 (launch)	7.3 Session 1
Breakpoint	5.4	6.1, 6.3	7.3 Sessions 1 and 2
Continue and step	5.4	6.1, 6.2	7.3 Sessions 1, 2
Inspect program	5.4, 5.5	6.1, 6.7	7.3 Sessions 1, 2
Trace system calls	5.4	6.1, 6.3	7.3 Sessions 3, 4
Two interfaces	5.1, 5.5	6.8	7.4 screenshots

1.5 Importance of the Application

The Mini Linux Debugger matters for four reasons: it teaches operating-system concepts, it builds practical debugging skill, it forms a base for more advanced tools, and it shows how much can be done with a few system calls.

Educational importance

Operating-system courses cover process creation, process states, signals, system calls and virtual memory. In this project all of these appear in a single program that students can run, step through and change. A student who watches RIP move from `0x401137` to `0x40113a` after one STEP understands the instruction pointer better than from any diagram.

Course topic	Where it appears in the debugger
Process creation (fork / exec)	`debugger_launch()`
Process states	Running, stopped, terminated (`running` and `stopped` flags)
Signals	SIGTRAP at breakpoints and steps; SIGKILL for STOP
System calls and kernel mode	System-call trace; every `ptrace()` request
CPU context switching	Registers saved by the kernel and read with `PTRACE GETREGS`
Virtual memory	Reading and patching the target's code with PEEK / POKE
Process synchronization	`waitpid()` after every resume request

Technical and practical importance

The techniques used here are the same as those inside GDB, `strace` and many security and profiling tools. Knowing them is a strong foundation for work in systems programming, reverse engineering, performance analysis and software security. The project also teaches good engineering habits: separating a reusable core from the user interface, checking every error return, and writing messages that explain what went wrong.

Importance as a starting point

Because the code is small and modular, it is a good starting point for further work. Extra features such as multiple breakpoints, memory writing, disassembly or symbol lookup can be added one at a time without redesigning the program, which makes the project suitable for later mini-projects and research exercises.

Importance in industry and research

Process tracing is not only a teaching topic. The same mechanism is used every day in professional work:

- **Software development.** Debuggers and system-call tracers are used to find the cause of

crashes and hangs in programs for which no source code or log is available.

- **Security analysis.** Analysts trace programs to see which files they open and which network connections they make, and use breakpoints when studying unknown software.
- **Operations and support.** Engineers attach tracers to misbehaving services to find which system call is blocking.
- **Operating-system and language-runtime development.** Developers of compilers, runtimes and kernels depend on debuggers to verify low-level behaviour.
- **Research.** Record-and-replay systems and dynamic analysis tools are built on the same operating-system interfaces.

Why build a debugger instead of only using one

Using a tool teaches how to use it; building a small version teaches how it works. The difference shows when something unexpected happens. A student who has written the breakpoint code knows immediately why the reported address is one byte off, why a breakpoint must be removed before stepping, and why a second breakpoint needs its own saved byte. A student who has only used GDB has to guess.

Value as a teaching example

Teaching need	How the project meets it
Short, complete example	About 2,100 lines in total, of which the core is a single file
Reproducible results	Fixed addresses (non-PIE target) and a Makefile that rebuilds everything
Visible cause and effect	Each command prints a message that names the operation performed
Room for extension	Clear list of limitations that can be turned into assignments (Section 8.1)

1.6 Target Users / Audience

The debugger is intended for people who want to learn how debugging works, rather than for people who need to debug large industrial programs.

Primary users

User group	What they need	How the project helps
Engineering students (OS, systems programming)	A hands-on example of processes, signals and system calls	Every feature maps to a visible system call; results can be checked against theory
Faculty and lab instructors	A short, understandable demonstration of `ptrace()`	Small source code, ready Makefile, sample target and sample sessions
Beginner programmers	A friendly way to see what a program does while it runs	GTK window with plain buttons: RUN, STEP, CONTINUE, REGISTERS
Developers and researchers	A minimal base for custom tracing tools	Clean separation of core and interface; easy to extend

Prerequisites for using the debugger

- Basic knowledge of C and the Linux command line.
- Understanding of what a process, a register and a memory address are.
- For breakpoints: the ability to find a function address with `objdump -d` or `gdb`.

Typical usage scenarios

1. **Classroom demonstration.** An instructor runs `my_debugger ./bin/test_program`, sets a breakpoint at `calculate()`, and shows the registers to explain argument passing (RDI and RSI hold 10 and 20).
2. **Lab experiment.** A student traces the first ten system calls of a program and compares the list with the output of `strace`.
3. **Self-study.** A beginner uses the GUI to step through a program one instruction at a time and watches RIP, RSP and RBP change.

Suggested learning path

The debugger can be used at several levels. A learner can start with the GUI and move step by step towards reading and changing the source.

Level	Activity	Concepts learnt
1 – Observe	Run the target, press STEP and REGISTERS repeatedly in the GUI	Instruction pointer, registers, stack pointer
2 – Experiment	Use the command line; set a breakpoint at `calculate()`; read memory	Breakpoints, addresses, memory contents

Level	Activity	Concepts learnt
3 – Understand	Read `debugger.c` alongside the outputs; follow each system call	ptrace, signals, wait status
4 – Extend	Add a command or remove a limitation from Section 8.1	Systems programming and design

Suggested lab exercises

1. **Find the arguments.** Set a breakpoint at `calculate()` and read RDI and RSI. Explain why they hold 10 and 20.
2. **Watch the stack frame.** Step through the first three instructions of `calculate()` and describe how RSP and RBP change.
3. **Count the system calls.** Run the trace once after launch and once after the breakpoint. Which calls come from program start-up and which from `printf`?
4. **Inspect the patch.** Read memory at the breakpoint address before and after setting the breakpoint. Which byte changed?
5. **Break at main.** Find the address of `main` with `objdump` and set the breakpoint there. What is RIP after the hit?
6. **Provoke an error.** Try an unreadable address and an unknown command; explain each message.

Limits of the intended audience

The debugger is not meant for debugging large production programs. It has no symbol support, one breakpoint, and no multi-threading, so developers needing those features should use GDB or LLDB. The project is best seen as a stepping stone towards those tools.

CHAPTER 2 OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

The Mini Linux Debugger is built directly on a small number of operating-system concepts. This section explains each concept and shows where it is used in the project.

Process creation: fork() and exec()

A new process is created with `fork()`, which makes a copy of the calling process. The copy then replaces its program image with a different one using `execl()`, which uses the `execve` system call. The debugger uses this fork-and-exec model so that the target is under control from its very first instruction: the child asks to be traced before it executes the target program.

The ptrace() system call

`ptrace()` is the Linux mechanism for process tracing. After a child calls `PTRACE_TRACEME`, the kernel lets its parent control it. Whenever the child receives a signal, it stops and the parent is notified. The parent can then read or write the child's registers and memory and decide how the child resumes: `PTRACE_CONT` (run freely), `PTRACE_SINGLESTEP` (run one instruction) or `PTRACE_SYSCALL` (run until the next system call boundary).

Signals and the INT3 trap

Signals are the way the kernel notifies a process of an event. The x86 instruction INT3 (opcode `0xCC`) is a one-byte instruction that causes a software trap; the kernel turns the trap into the signal `SIGTRAP`. Single-stepping also ends with `SIGTRAP`, because the processor can be told to trap after each instruction. `SIGKILL` cannot be caught and is used to terminate the target.

Waiting for a child: waitpid()

The parent calls `waitpid()` to wait until the traced child changes state. The returned status is decoded with the macros `WIFEXITED`, `WIFSIGNALED` and `WIFSTOPPED`, which tell the debugger whether the program finished, was killed by a signal, or merely stopped. `WSTOPSIG` gives the signal that caused a stop.

CPU context and registers

When a process is stopped, its CPU state (general-purpose registers, the instruction pointer RIP, the stack pointer RSP and the flags register) is saved by the kernel. `PTRACE_GETREGS` copies this state into a `struct user_regs_struct`, and `PTRACE_SETREGS` writes it back. The debugger uses this to display registers and to move RIP back by one byte after a breakpoint.

Virtual memory

Each process has its own virtual address space, which the debugger cannot access directly. It

asks the kernel to read and write the target's memory one machine word (8 bytes) at a time with `PTRACE_PEEKDATA` and `PTRACE_POKEDATA`. Writing a byte into the program's code area is exactly how a breakpoint is inserted.

System calls and user / kernel mode

A program asks the kernel for services such as reading a file or allocating memory through system calls, which switch the CPU from user mode to kernel mode. With `PTRACE_SYSCALL` the kernel stops the traced process at the **entry** and at the **exit** of every system call. On x86-64 Linux the call number is in `orig_rax` at entry and the return value is in `rax` at exit.

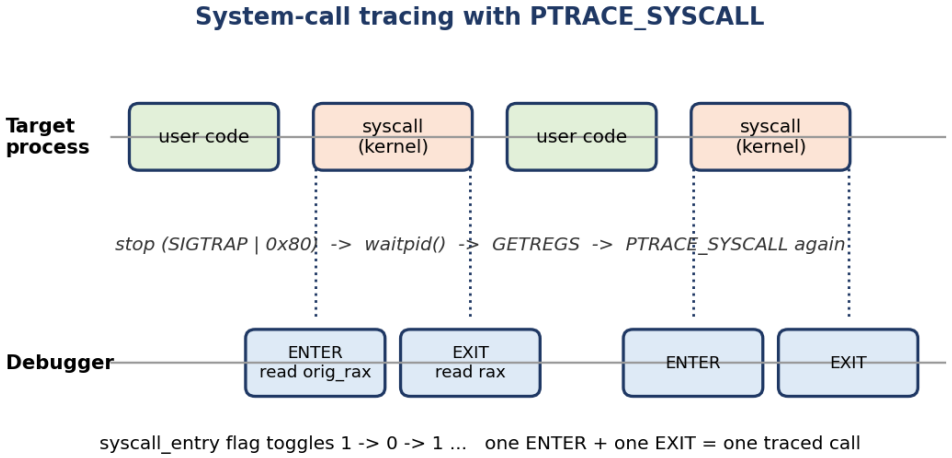


Figure 2.1: How PTRACE_SYSCALL exposes the entry and exit of system calls

Concept	System call / interface	Used in
Process creation	`fork()`, `execl()`	Launch
Tracing	`ptrace()`	Every module
Notification	`waitpid()`, `SIGTRAP`	Every run-control command
CPU context	`PTRACE_GETREGS` / `SETREGS`	Registers, breakpoint step-over
Memory	`PTRACE_PEEKDATA` / `POKEDATA`	Breakpoint, memory read
Termination	`kill()`, `SIGKILL`	Stop

2.2 Existing Methods / Algorithms

Several well-known methods exist for finding bugs and for observing running programs. Each has strengths, and each helped to shape the design of this project.

Tools and methods in common use

Tool / method	How it works	Limitation for learning
GDB	Full source-level debugger using ptrace, symbol tables, DWARF data and CPU debug registers	Very large and complex; internal working is hidden from beginners
LLDB	Debugger of the LLVM project with similar capabilities	Same complexity; designed for large projects
strace	Traces system calls of a process using `PTRACE_SYSCALL`	Only traces system calls; no breakpoints, stepping or registers
ltrace	Traces calls to shared-library functions	Library calls only; no execution control
Valgrind	Runs the program on a synthetic CPU to detect memory errors	Heavyweight; not an interactive debugger
printf debugging	Programmer inserts print statements and re-runs	Needs source changes; cannot pause or inspect state
Core-dump analysis	Examines the saved memory image after a crash	Post-mortem only; cannot watch the program run
Record and replay (e.g. rr)	Records an execution and replays it deterministically	Advanced; relies on ptrace and hardware counters

Software breakpoints versus hardware breakpoints

A **software breakpoint** replaces the first byte of an instruction with INT3. When the CPU reaches it, it traps, the kernel stops the process, and the debugger regains control. The original byte is kept by the debugger and put back when needed. There is no limit on the number of software breakpoints, and they work on any x86 processor.

A **hardware breakpoint** uses the CPU's debug registers (DR0–DR3), which hold up to four addresses. The processor itself traps when execution (or a data access) reaches one of those addresses, so the program's code is not modified. Hardware breakpoints can also watch data, but only four are available and they must be configured through `ptrace()`. Debuggers use both kinds.

The Mini Linux Debugger uses software breakpoints because they show the whole mechanism clearly: read a word, change one byte, write it back, trap, restore.

Methods of stepping and tracing

- **Single-stepping** asks the CPU to trap after one instruction; the kernel exposes it as `PTRACE_SINGLESTEP`.

- **System-call tracing** with `PTRACE_SYSCALL` stops the target at each system-call entry and exit; this is what `strace` is built on.
- **Reading state** is done with `PTRACE_GETREGS` for registers and `PTRACE_PEEKDATA` for memory; an alternative for memory is reading `/proc/<pid>/mem`.

How the project compares

Capability	GDB	strace	Mini Linux Debugger
Start a program under control	Yes	Yes	Yes
Breakpoints	Many, by name or address	No	One, by address
Single step	Yes	No	Yes
Registers and memory	Yes	No	Yes (all registers; one word)
System-call trace	Limited (`catch syscall`)	Yes	Yes (10 calls per request)
Source-level debugging	Yes	No	No
Small, readable source	No	No	Yes (about 1,000 lines core)

2.3 Proposed Approach

The proposed approach is to implement the essential operations of a debugger directly on top of Linux system calls, in one reusable C module, and to attach two different user interfaces to it. The design follows the order in which a debugging session actually happens.

Step-by-step approach

1. **Launch.** The debugger calls `fork()`. The child calls `ptrace(PTRACE_TRACEME)` and then `execl()` to run the target. The child stops at the exec, and the parent waits for it with `waitpid()` and then enables `PTRACE_O_TRACESYSGOOD`.
2. **Breakpoint.** The original word at the chosen address is read and saved. Its lowest byte is replaced by `0xCC` (INT3) and the word is written back.
3. **Continue / step.** If the program is stopped on the breakpoint, the original byte is restored, RIP is moved back by one, one instruction is single-stepped, and the breakpoint is re-installed. Then execution continues with `PTRACE_CONT`.
4. **Inspect.** Registers are read with `PTRACE_GETREGS` and memory with `PTRACE_PEEKDATA`.
5. **Trace system calls.** `PTRACE_SYSCALL` is used repeatedly; with `PTRACE_O_TRACESYSGOOD` the system-call stops can be told apart from ordinary SIGTRAP stops. Entry and exit are reported alternately.
6. **Report.** Every operation writes a text message into a buffer, which the command-line program prints or the GUI shows in its Debug Output pane.

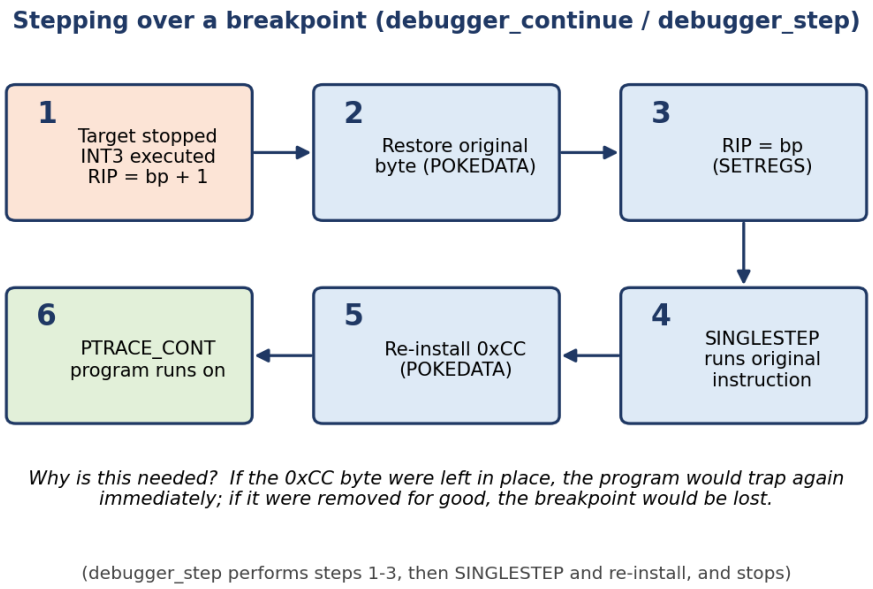


Figure 2.2: Stepping over a breakpoint

Key design decisions

Decision	Alternative	Reason for the choice
Start the target ourselves (fork + exec)	Attach to a running process	Simpler and needs no special permission
Software breakpoint (INT3)	Hardware debug registers	Shows the whole mechanism and has no limit of four
One shared core, two front ends	Separate code for CLI and GUI	No duplicated logic; both behave identically
Messages written to a caller-supplied buffer	Printing directly inside the core	The same core works in a terminal and in a window
Check `running` and `stopped` before each command	Trust the caller	Prevents illegal operations and gives clear errors

2.4 Advantages of the Proposed Approach

The approach has been chosen to give a learner the clearest possible view of how debugging works. Its advantages are summarised below.

Advantages

- **Small and readable.** The whole engine is one C source file and one header, so the complete flow can be followed from the first line to the last.
- **Transparent.** Every feature maps to a visible system call. There is no hidden library doing the work.
- **Reusable design.** The engine contains no user-interface code. The CLI and the GUI call the same functions with the same message-buffer interface.
- **Safe operation.** Each function first checks that the program is running and stopped, and reports a clear message if it is not.
- **Minimal dependencies.** The command-line version needs only GCC and the Linux kernel; GTK+ is needed only for the GUI.
- **Correct breakpoint handling.** The step-over procedure means the program continues normally after a breakpoint instead of crashing or trapping forever.
- **Easy to extend.** Further commands such as multiple breakpoints, memory writing or symbol lookup can be added without changing the structure.

Comparison with the alternatives

Aspect	Large debugger (e.g. GDB)	Mini Linux Debugger
Size of source code	Hundreds of thousands of lines	About 2,100 lines in total
Time to understand	Weeks or months	A few hours
Visibility of OS calls	Hidden behind commands	Each call is visible in the code
Features	Very complete	Essential features only
Suitable for learning	Partly	Yes – designed for it

Honest limitations

The advantages come with trade-offs. The debugger supports only one breakpoint, reads one word at a time, shows no source lines, and knows only 24 system-call names. These limitations are acceptable for a teaching tool, and they are listed in Section 8.3 together with ideas for removing them.

Evaluation against the design goals

Goal	How the approach satisfies it	Evidence

Goal	How the approach satisfies it	Evidence
Learnability	One small core; one system call per feature	Section 6.4 maps every feature to a call
Correctness	Step-over procedure for breakpoints	Program prints `Result = 60` after a breakpoint (7.3)
Robustness	State checks and error messages	15 test cases pass, including faulty inputs
Reusability	Message-buffer interface	CLI and GUI share `debugger.c`
Low cost	Free tools, no special hardware	Runs on any x86-64 Linux computer

Trade-offs of the approach

Every design choice gives something up. Starting the target ourselves means the debugger cannot examine a program that is already running. Software breakpoints change the program's code while the breakpoint is installed, which a hardware breakpoint would avoid. Returning messages as text keeps the core simple, but it means the front end cannot easily format values in its own way. These trade-offs were accepted because the project's goal is clarity.

Suitability for laboratory use

- The whole project builds in a few seconds with `make`.
- No administrator rights are needed on a normal Ubuntu installation.
- The sample target gives identical code addresses on every machine with the same compiler, so a lab manual can quote addresses.
- Because the sources are short, students can modify them during a single lab session.

CHAPTER 3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The Mini Linux Debugger is a very small program and runs on modest hardware. The processor type matters more than speed or memory, because the debugger works with x86-64 registers and the x86 INT3 instruction.

Hardware needed

Component	Minimum	Recommended
Processor	64-bit Intel or AMD (x86-64)	Dual-core x86-64 or better
RAM	2 GB	4 GB or more
Storage	500 MB free	1 GB free (tools, GTK libraries and project)
Display	Not required for the CLI version	1280 × 720 or higher for the GUI
Input devices	Keyboard	Keyboard and mouse

Why an x86-64 processor is required

Three parts of the debugger depend on the processor architecture:

- **Register set.** `struct user_regs_struct` and the register names RAX, RBX, … R15, RIP and EFLAGS are those of x86-64.
- **Breakpoint instruction.** The one-byte opcode `0xCC` is INT3 on x86. Other processors use different breakpoint instructions of different lengths.
- **System-call convention.** The call number is read from `orig_rax`, and the numbers in `syscall_name()` (for example 1 = write, 231 = exit_group) are the x86-64 Linux numbers. On ARM64 the numbers are different.

The debugger will therefore not work correctly on ARM-based computers without code changes.

Resources used by the debugger

The debugger itself needs very little. The executables built from the project are small, and the memory used by the debugger is mostly its fixed-size message buffers.

Item	Size
`bin/my debugger` (CLI)	about 41 KB
`bin/debugger gui` (GUI)	about 61 KB (plus the shared GTK libraries)
`bin/test program` (target)	about 17 KB
Message buffer in CLI and GUI	4096 bytes
Message buffer for system-call trace	8192 bytes

The only processor feature used beyond ordinary execution is the trap flag that makes the CPU stop after one instruction, which every x86-64 processor provides.

3.2 Software Requirements

The project needs a Linux system, a C compiler, the Make tool and, for the graphical interface only, the GTK+ 3 development files. All of them are available free of charge from the Ubuntu package repositories.

Required software

Software	Purpose
Ubuntu 22.04 / 24.04 LTS (or WSL2 with Ubuntu)	Operating system providing `ptrace`, `fork` and signals
GCC (C compiler)	Compiles the debugger, the GUI and the sample target program
GNU Make	Builds everything using the supplied Makefile
libgtk-3-dev and pkg-config	GTK+ 3 headers and libraries for the graphical front end
objdump (binutils)	Finding instruction addresses for breakpoints
GDB (optional)	Cross-checking results against a professional debugger
Text editor or VS Code	Editing the source code

Installation

On Ubuntu all required packages can be installed with two commands:

```
sudo apt update
sudo apt install build-essential make pkg-config libgtk-3-dev gdb
```

The package `build-essential` supplies GCC, `make` and the standard C library headers. `libgtk-3-dev` brings in GTK+ 3 together with its dependencies, and `pkg-config` tells the compiler where to find them.

Checking the installation

```
gcc --version
make --version
pkg-config --modversion gtk+-3.0
objdump --version | head -1
uname -m          # should print x86_64
```

If `pkg-config` reports that `gtk+-3.0` is not found, the GUI cannot be built, but the command-line debugger (`make debugger`) still builds without GTK.

Roles of the software components

Component	Needed for	Used by
Linux kernel	`ptrace`, signals, process control	Debugger core
glibc and kernel headers	`sys/ptrace.h`, `sys/user.h`, `sys/wait.h`	Debugger core
GTK+ 3	Window, buttons, text view	`gui.c` only

Component	Needed for	Used by
GCC and Make	Building	Developer
objdump / GDB	Finding addresses, checking results	User of the debugger

3.3 Operating System Environment – Ubuntu / WSL

The project is written for Linux and has been built for Ubuntu. It uses Linux-specific interfaces (`ptrace`, `sys/user.h` and the x86-64 system-call numbering), so it does not run natively on Windows or macOS. This section describes the environments in which it can be used.

Supported environments

Environment	Suitability	Notes
Ubuntu on a physical computer	Best	Everything works, including the GUI
Ubuntu in a virtual machine	Very good	Needs an x86-64 guest; GUI works through the VM window
WSL2 with Ubuntu (Windows 11 / 10)	Good	Real Linux kernel; GTK window shown through WSLg
WSL1	Not recommended	Incomplete `ptrace` support
Docker container	Possible	May need `--cap-add=SYS_PTRACE` because of the default seccomp profile
macOS or native Windows	Not supported	No Linux `ptrace`

Permissions

Because the debugger starts its own target and the child calls `PTRACE_TRACEME`, no administrator rights are normally needed. Linux can restrict `ptrace` through the Yama security module; its setting can be checked with:

```
cat /proc/sys/kernel/yama/ptrace_scope
```

The default value (1) restricts only attaching to unrelated processes, which this debugger never does. A value of 3 disables `ptrace` completely and would stop the debugger from working.

Setting up WSL2

1. Open PowerShell as administrator and run `wsl --install -d Ubuntu`.
2. Restart the computer if requested and create the Ubuntu user name and password.
3. Inside Ubuntu, install the packages shown in Section 3.2.
4. Copy or clone the project into the Linux file system (for example the home directory), not into `/mnt/c`.
5. Run `make` and start the debugger. A GUI window opens on the Windows desktop through WSLg.

Common environment problems

Symptom	Likely cause	Remedy

Symptom	Likely cause	Remedy
"PTRACE_TRACEME: Operation not permitted"	Container or Yama setting blocks `ptrace`	Run outside the restricted container or add `SYS_PTRACE`
GUI window does not open	No display (SSH without forwarding, WSL without WSLg)	Use the CLI or enable a display
Breakpoint address "Failed to read memory"	Wrong address or program is a PIE build	Rebuild target with `-no-pie` and re-check address

3.4 Development Tools and Compilers

The project is built with GCC and GNU Make. The supplied Makefile has a target for each executable, so the whole project is built with one command.

The Makefile

```
CC = gcc
CFLAGS = -Wall -Wextra -g -O0 -Iinclude
TARGET_FLAGS = -no-pie
GTK_FLAGS = $(shell pkg-config --cflags --libs gtk+-3.0)

all: test_program debugger gui

test_program:
     $(CC) $(CFLAGS) $(TARGET_FLAGS) test/test_program.c -o bin/test_program
debugger:
     $(CC) $(CFLAGS) src/debugger.c src/main.c -o bin/my_debugger
gui:
     $(CC) $(CFLAGS) src/debugger.c gui/gui.c -o bin/debugger_gui $(GTK_FLAGS)
```

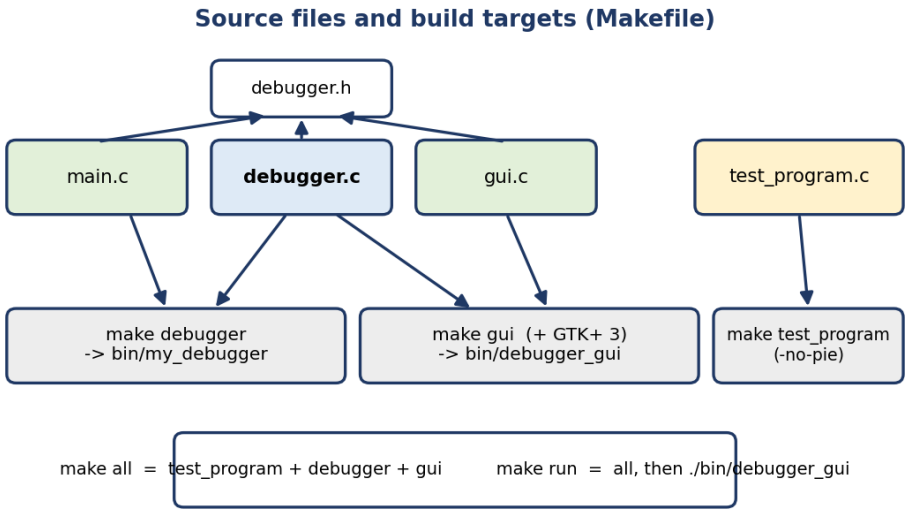


Figure 3.1: Source files and build targets

Compiler options

Flag	Meaning
`-Wall -Wextra`	Enable common and extra compiler warnings to catch mistakes early
`-g`	Include debugging information in the executable
`-O0`	Disable optimisation so the generated code matches the source and is easy to follow
`-Iinclude`	Add the `include` directory to the header search path
`-no-pie` (target program only)	Build the target at fixed addresses so breakpoint addresses are the same on every run
`pkg-config --cflags --libs gtk+-3.0`	Supplies the compiler and linker options for GTK+ 3 (GUI only)

Companion tools

Tool	Use in the development workflow
`objdump -d`	Disassemble the target to find the address of `calculate()` and `main()`
`gdb`	Compare addresses, registers and behaviour with a standard debugger
`strace`	Verify the system-call sequence reported by the debugger
`file`	Check whether an executable is PIE or non-PIE

CHAPTER 4 TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

The project is written entirely in the **C** language. C was chosen for three reasons. First, the Linux system-call interface, `ptrace()` and the kernel structures such as `struct user_regs_struct` are defined for C. Second, C gives direct control over pointers, addresses and individual bytes, which is exactly what a debugger needs. Third, C code maps closely to the machine instructions it produces, which keeps the link between the source and the operating-system behaviour clear. C++ is not used.

C features used in the project

C feature	Where it is used
`struct` (the `Debugger` type)	Holds pid, flags, breakpoint address and saved word (`debugger.h`)
Pointers and casts between integers and addresses	`(void *)addr` passed to `PTRACE_PEEKDATA` / `POKEDATA`
Bitwise operations	Replacing the lowest byte with `0xCC`: `(original & ~0xFFULL) | 0xCCULL`
`static` functions	Private helpers `msg`, `wait_for_child`, `read_word`, `write word`
Header file	`debugger.h` separates the public interface from `debugger.c`
`snprintf()`	Bounded formatting of every message to avoid buffer overflows
`switch` statement	`syscall name()` converts a number into a name
Macros	`WIFEXITED`, `WIFSTOPPED`, `WSTOPSIG` decode the wait status

Example: bit manipulation for the breakpoint

A breakpoint changes only one byte of a word that is read and written as a whole. The statement below clears the lowest byte with a mask and then sets it to `0xCC`:

```
    unsigned long long patched =
        ((unsigned long long)original & ~0xFFULL) | 0xCCULL;
```

Coding practices followed

- Every function that can fail returns 0 for success and -1 for failure, and also fills the message buffer.
- The return value of every system call is checked.
- Fixed buffers are written only with `snprintf()` so that output can never overflow them.
- The program compiles without warnings under `-Wall -Wextra`.

Structure of the code base

The code is organised so that each file has one job. The command-line front end, for example, is a simple loop that reads a line and passes it to the core. The start of its loop is shown below.

```
    while (dbg.running) {
        printf("\n(debugger) ");
        fflush(stdout);
        char command[100];
        if (!fgets(command, sizeof(command), stdin))
            break;
        /* Continue */
        if (command[0] == 'c') {
            debugger_continue(
                &dbg,
                message,
                sizeof(message)
            );
        }
```

Each command is a short `if` / `else if` block that calls one core function and then prints the returned message. This structure is easy to extend: a new command needs a new block and a new line in the help text.

Error-handling style

Errors are handled in the same way everywhere: test the result of the call, write a message, return -1. The example below is the complete `debugger_stop()` function.

```
int debugger_stop(Debugger *dbg,
                  char *message,
                  size_t message_size)
{
    if (dbg == NULL || !dbg->running) {
        msg(message, message_size,
            "No running program.");
        return -1;
    }
    if (kill(dbg->pid, SIGKILL) == -1) {
        msg(message, message_size,
            "Failed to stop program.");
        return -1;
    }
    int status;
    if (wait_for_child(dbg, &status) == -1) {
        msg(message, message_size,
            "waitpid() failed.");
        return -1;
    }
    dbg->running = 0;
    dbg->stopped = 0;
    dbg->breakpoint_set = 0;
    msg(message, message_size,
        "Program stopped.");
    return 0;
}
```

Why not C++

C++ would have offered classes and strings, but it would hide some of what the project wants to show: plain structures, explicit buffers, and direct system calls. The `Debugger` structure plays

the role of an object, and the message-buffer convention replaces exceptions, which keeps every step explicit.

4.2 GCC Compiler

GCC (the GNU Compiler Collection) compiles all source files in the project. It performs four stages for each file, usually in one command.

The compilation stages

Stage	What happens	Result
Preprocessing	`#include` files are inserted and macros are expanded	Expanded C source
Compilation	C is translated into assembly language	Assembly code
Assembly	Assembly is converted into machine code	Object code
Linking	Object files and libraries are combined	Executable file

How each executable is produced

Executable	Sources	Notes
`bin/my_debugger`	`src/debugger.c`, `src/main.c`	No external library
`bin/debugger_gui`	`src/debugger.c`, `gui/gui.c`	Linked with GTK+ 3
`bin/test_program`	`test/test_program.c`	Built with `-no-pie`

Why the target is built with -no-pie

Modern Linux systems build executables as position-independent (PIE) by default, which means the program is loaded at a different address every time. A breakpoint address typed by the user would then be wrong in the next run. With `-no-pie` the code of `test_program` is always loaded at the same addresses; for example `calculate()` is always at `0x401136`. The `file` command shows the difference: the target is reported as a plain executable, whereas the debugger itself is a PIE executable.

Why optimisation is disabled

The option `-O0` keeps the generated machine code close to the source. With optimisation, GCC could remove the variable `result` or inline `calculate()`, and the demonstration would no longer match the source code. The compiler warnings `-Wall -Wextra` were also used throughout development, and the debugger and command-line front end compile without any warning.

Inspecting the built programs

Simple commands reveal how the executables were linked and how large they are. The debugger uses only the C library, while the GUI additionally needs the GTK libraries.

```
$ ldd bin/my_debugger
    linux-vdso.so.1
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6
    /lib64/ld-linux-x86-64.so.2

$ size bin/my_debugger bin/test_program
   text    data     bss     dec     hex  filename
  15016     720      32   15768    3d98  bin/my_debugger
   1170     544       8    1722     6ba  bin/test_program
```

The `text` column is the size of the machine code. The sample target has only 1170 bytes of code, and the whole debugger about 15 KB, which fits the aim of a small, readable project.

Effect of the main compiler options

Option	Without it	With it
`-Wall -Wextra`	Suspicious code compiles silently	The compiler reports unused variables, missing returns and sign problems
`-g`	No names or line numbers in the executable	Debug information is included, so GDB can show source lines when used for checking
`-O0`	Optimiser may remove or merge code	Code matches the source; the sample behaves as described in this report
`-no-pie`	Code is loaded at a random address	Code of the target is always at the same address

Compiling by hand

Instead of `make`, each program can be compiled with one command. This is useful to see what the Makefile does for each target.

```
gcc -Wall -Wextra -g -O0 -Iinclude -no-pie test/test_program.c -o bin/test_program
gcc -Wall -Wextra -g -O0 -Iinclude src/debugger.c src/main.c -o bin/my_debugger
```

4.3 Linux System Calls

The debugger is almost entirely a carefully ordered collection of system calls. The table lists those used, and the sections that follow explain the most important ones.

System calls used

System call	Used for
`fork()`	Creates the child process that will become the target program
`execl()` (execve)	Replaces the child with the target program
`ptrace()`	Controls the target: TRACEME, SETOPTIONS, CONT, SINGLESTEP, SYSCALL, GETREGS, SETREGS, PEEKDATA, POKEDATA
`waitpid()`	Waits until the target stops, exits or is killed and returns its status
`kill()`	Sends `SIGKILL` to terminate the target (STOP command)

ptrace(request, pid, addr, data)

`ptrace()` is a single system call whose first argument selects the operation. The table shows each request used in the project.

Request	Effect
`PTRACE_TRACEME`	Child asks to be traced by its parent
`PTRACE_SETOPTIONS`	Sets options; `PTRACE_O_TRACESYSGOOD` marks system-call stops as `SIGTRAP | 0x80`
`PTRACE_PEEKDATA`	Reads one word from the target's memory
`PTRACE_POKEDATA`	Writes one word to the target's memory
`PTRACE_GETREGS` / `PTRACE_SETREGS`	Reads / writes all general-purpose registers
`PTRACE_CONT`	Resumes the target
`PTRACE_SINGLESTEP`	Resumes the target for exactly one instruction
`PTRACE_SYSCALL`	Resumes the target until the next system-call entry or exit

waitpid(), kill() and the exec family

`waitpid(pid, &status, 0)` blocks until the target changes state and stores a status word that is decoded with macros. `kill(pid, SIGKILL)` terminates the target at once. `execl(program, program, NULL)` loads the target program in place of the child; on success it never returns.

System-call names known to the trace

The function `syscall_name()` recognises 24 common x86-64 call numbers. Any other number is shown as "unknown".

Number	Name	Number	Name	Number	Name

Number	Name	Number	Name	Number	Name
0	read	12	brk	62	kill
1	write	39	getpid	63	uname
2	open	41	socket	89	readlink
3	close	42	connect	158	arch_prctl
4	stat	57	fork	231	exit_group
5	fstat	59	execve	257	openat
6	lstat	60	exit	9 / 10 / 11	mmap / mprotect / munmap
		61	wait4		

4.4 POSIX Threads / Synchronization

POSIX threads (`pthread`) allow one program to run several flows of control at the same time. They are **not used** in this project: the debugger is a single-threaded program. This section explains how correct synchronization is still achieved and why threads were not needed.

Synchronization without threads

The debugger and its target are two separate processes that must never work on the same state at the same time. In particular, the debugger must not read registers or memory while the target is running. This is guaranteed by a strict pattern: after every request that resumes the target (`PTRACE_CONT`, `PTRACE_SINGLESTEP`, `PTRACE_SYSCALL`) the debugger immediately calls `waitpid()`, which blocks until the target stops again. Only then does the debugger continue.

```
ptrace(PTRACE_CONT, pid, NULL, NULL);   /* target runs        */
waitpid(pid, &status, 0);               /* debugger waits     */
/* only now may registers / memory be read */
```

Guards in the Debugger structure

The flags `running` and `stopped` give a second level of protection. Every command checks them first and refuses to act in the wrong state, with messages such as "Program must be stopped before single stepping." and "Program is already running."

Mechanism	Purpose
`waitpid()` after each resume	Debugger and target never run concurrently on shared state
`running` flag	Rejects commands when there is no target
`stopped` flag	Rejects inspection while the target is running
`syscall_entry` flag	Pairs each ENTER with its EXIT during tracing

The GUI and threads

The GTK front end also uses one thread: the GTK main loop. While the debugger waits for the target, the window cannot redraw. For the short test program this is not noticeable. A long-running target would freeze the window until it stops.

Possible future use of threads

A future version could run the debugger functions in a worker thread and send results back to the GTK main loop, for example with `g_idle_add()`. That would keep the window responsive during a long run, but it would then need a mutex around the `Debugger` structure. Because ptrace requires that all requests for a traced process come from the same thread, the worker thread would have to be the one that created the target.

ptrace and threads

Linux ties a trace relationship to a particular thread: the `ptrace` requests for a target must be made by the thread that is its tracer. A multi-threaded debugger must therefore keep all debugging calls in one thread. This rule is one reason why the project avoids threads: with a single thread it is automatically satisfied.

Hazards that threads would introduce

Hazard	Description	How this design avoids it
Data race on `Debugger`	Two threads changing flags at the same time	Only one thread exists
Reading while running	Registers read while the target runs	`waitpid()` always follows a resume request
Wrong tracer thread	`ptrace` call from a different thread	All calls come from the main thread
Lost output	Two commands writing to the same buffer	Commands run one after another

Sketch of a threaded GUI (future work)

The outline below is a design idea and is **not implemented** in the project. The GUI would send commands to a worker thread that owns the target and all `ptrace` calls, and the worker would send each result back to the GTK main loop.

```
GTK main thread                     Worker (debugger) thread
---------------                     ------------------------
button clicked  --- command --->    take command from queue
                                    call debugger_*() function
update text view <-- g_idle_add --  send message back
window stays responsive            (may block in waitpid)
```

A mutex or queue would protect the command list. This design would stop the window freezing during a long run, at the cost of more code and more places where a mistake could occur.

4.5 Linux Commands and Utilities

The project is built, run and checked with a handful of standard Linux commands. The table lists them with their purpose.

Commands used

Command	Purpose in this project
`make`, `make clean`, `make run`	Build, clean and run the project using the Makefile
`gcc`	Compile the sources manually when required
`objdump -d bin/test_program`	Disassemble the target to find addresses for breakpoints
`gdb`	Cross-check addresses, registers and behaviour
`strace ./bin/test_program`	Compare the real system-call sequence with the trace
`file`	Check whether an executable is PIE or non-PIE
`ps`	See the running target process and its PID
`cat /proc/sys/kernel/yama/ptrace_scope`	Check whether the kernel allows `ptrace`

Finding a breakpoint address with objdump

The debugger needs an address, so the first step of every session is to find one. `objdump` shows the address of each instruction of the target. For `test_program` the relevant part is:

```
$ objdump -d bin/test_program --no-show-raw-insn
0000000000401136 <calculate>:
  401136:  endbr64
  40113a:  push   %rbp
  40113b:  mov    %rsp,%rbp
  40113e:  mov    %edi,-0x14(%rbp)
  ...
0000000000401157 <main>:
  401157:  endbr64
  40115b:  push   %rbp
```

The address on the left of `<calculate>:` (here `0x401136`) is typed into the debugger as the breakpoint address.

Typical workflow

1. Build everything with `make`.
2. Disassemble the target with `objdump -d` and note the address of the function of interest.
3. Start `./bin/my_debugger ./bin/test_program` (or the GUI).
4. Set the breakpoint, continue, and inspect registers and memory.
5. Optionally run `strace` or `gdb` on the same program to confirm the results.

Useful companion experiments

The commands below are not part of the debugger but help to verify what it reports.

Experiment	Command	What to look for
Compare system calls	`strace ./bin/test_program`	The same calls as the trace, with `write` of 12 bytes
Check the code at a breakpoint address	`objdump -d bin/test_program --start-address=0x401136 --stop-address=0x401140`	`endbr64` (bytes `f3 0f 1e fa`) at 0x401136, which the debugger temporarily changes to `cc`
See the process while stopped	`ps -o pid,ppid,stat,cmd`	The target is a child of the debugger
Check ptrace permission	`cat /proc/sys/kernel/yama/ptrace_scope`	Value 0, 1 or 2 allows the debugger; 3 forbids it
Check build type	`file bin/test_program`	Plain executable, not PIE

Reading objdump output

Each line of `objdump -d` has three parts: the address, optionally the raw bytes, and the instruction. The debugger uses the address. The raw bytes (shown with `objdump -d` without `--no-show-raw-insn`) are what the debugger reads and patches. At 0x401136 they are `f3 0f 1e fa`, the four bytes of `endbr64`; the breakpoint replaces only the first of them with `cc`.

Quick reference

```
make                        build everything
make debugger               build only the command-line debugger
objdump -d bin/test_program   find addresses
./bin/my_debugger ./bin/test_program   start a session
file bin/test_program       check PIE / non-PIE
```

4.6 Other Tools / Libraries Used

Besides GCC and the kernel interface, the project uses the GTK+ 3 graphical toolkit, the GNU C library and a few Linux header files.

GTK+ 3

GTK+ is a C toolkit for graphical interfaces. `gui.c` builds the window with the `GtkApplication` model and connects button clicks to functions with `g_signal_connect()`.

Widget (as used in gui.c)	Purpose
`GtkApplication` / `gtk_application_window_new`	Application object and main window
`GtkBox` (vertical and horizontal)	Layout of rows
`GtkLabel`	Title, status and field captions
`GtkEntry`	Program path, breakpoint address, memory address
`GtkButton`	RUN, CONTINUE, STEP, REGISTERS, SYSTEM CALL TRACE, STOP, SET BREAKPOINT, READ MEMORY
`GtkFrame` + `GtkScrolledWindow`	Framed, scrollable area for output
`GtkTextView` (read-only, monospace)	Debug Output pane

GNU C library and Linux headers

Header	Provides
`sys/ptrace.h`	`ptrace()` and the `PTRACE_*` request constants
`sys/user.h`	`struct user_regs_struct` (register layout)
`sys/wait.h`	`waitpid()` and the `W*` status macros
`signal.h`	`SIGTRAP`, `SIGKILL` and `kill()`
`unistd.h`	`fork()`, `execl()`, `_exit()`
`errno.h`	`errno`, used to detect failed memory reads
`stdio.h`, `stdlib.h`, `string.h`	`snprintf`, `strtoull`, `memset`

GNU Make

Make reads the Makefile and runs the right compiler command for each target, so that `make` builds everything and `make clean` removes the built files. The `run` target builds all programs and starts the GUI.

GTK functions used in gui.c

Function	Purpose
`gtk_application_new` / `g_application_run`	Create the application and start the event loop

Function	Purpose
`gtk application window new`, `gtk_window_set_title`, `gtk_window_set_default_size`	Create and configure the window
`gtk_box_new`, `gtk_box_pack_start`	Arrange widgets in rows
`gtk entry new`, `gtk entry get text`, `gtk_entry_set_text`	Text fields for path and addresses
`gtk_button_new_with_label`, `g signal connect`	Create buttons and attach callbacks
`gtk_text_buffer_insert`, `gtk text buffer get end iter`	Append text to the output pane
`gtk_label_set_text`	Update the status label
`gtk_widget_show_all`	Display the window and its contents

How the GTK event loop works

After `g_application_run()` is called, GTK waits for events such as button clicks. When the user clicks a button, GTK calls the callback that was connected to it. The callback runs to completion, and only then does GTK handle the next event. This is why a callback that waits for the target also blocks the window.

C library functions used

Function	Purpose in the project
`fork`, `execl`, `_exit`	Create the target process and end the child on failure
`waitpid`, `kill`	Wait for and terminate the target
`snprintf`	Safe formatting of every message
`strtoull`	Convert typed addresses to numbers
`fgets`, `puts`, `printf`, `fflush`	Command-line input and output
`memset`, `perror`	Clear the structure; report errors in the child

CHAPTER 5 SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The Mini Linux Debugger has a layered design. The user works through one of two front ends. Both front ends call the same debugger core, the core talks to the Linux kernel through system calls, and the kernel controls the target process. Figure 5.1 shows the layers.

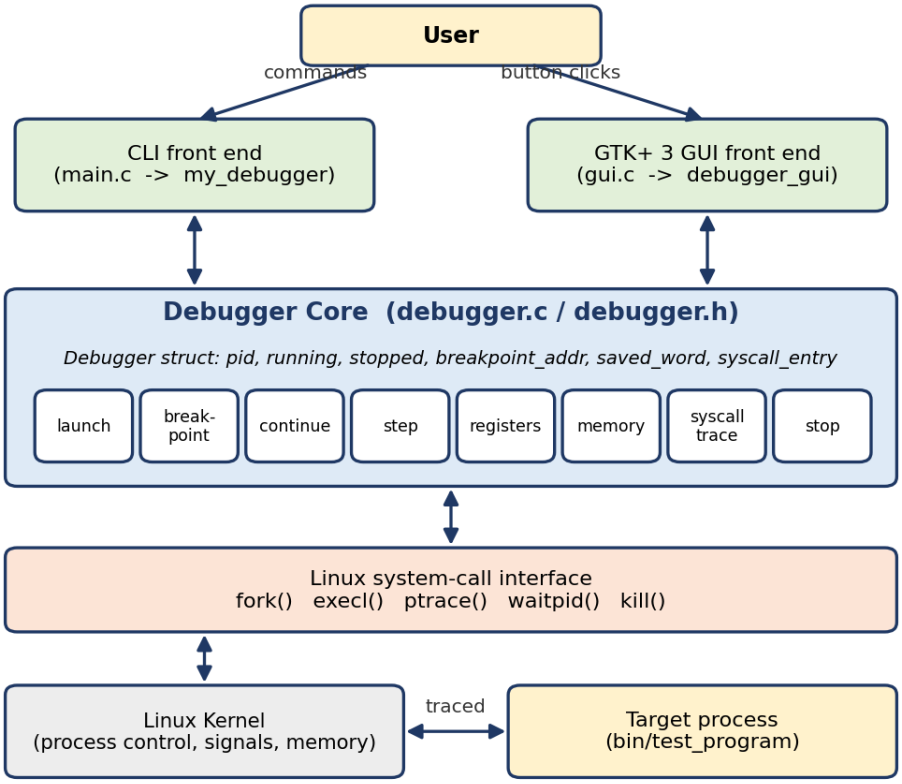


Figure 5.1: Overall architecture of the Mini Linux Debugger

Layers and their responsibilities

Layer	Component	Responsibility
User interface	`main.c` (CLI) and `gui.c` (GTK+ 3)	Read the user's input, validate it, call the core, and display the returned message
Debugger core	`debugger.c` and `debugger.h`	All debugging logic: launch, breakpoints, run control, registers, memory, system-call trace
System-call interface	`fork`, `execl`, `ptrace`, `waitpid`, `kill`	The only way the core affects the target process
Kernel	Linux	Enforces tracing rules, stops and resumes the target, delivers signals
Target process	`bin/test_program`	The program being debugged

The Debugger structure

The complete state of a debugging session is held in one structure, `Debugger`, declared in `debugger.h`. Every core function receives a pointer to it, so the core itself needs no global

variables.

Field	Meaning
`pid`	Process ID of the target (child) process
`running`	1 while the target process exists, 0 after it exits or is killed
`stopped`	1 while the target is paused and can be inspected
`breakpoint_set`	1 if a breakpoint is currently installed
`breakpoint_addr`	Address where the INT3 byte was written
`saved_word`	Original 8-byte word at that address, kept so it can be restored
`syscall_tracing`	Flag showing that system-call tracing is in progress
`syscall_entry`	Toggles between system-call entry (1) and exit (0)

Design principles

- **Separation of concerns.** The core contains no input or output code; front ends contain no debugging logic.
- **One uniform interface.** Every core function has the form `int function(Debugger *dbg, ..., char *message, size_t message_size)` and returns 0 for success or -1 for failure.
- **State checks first.** Each function begins by checking `running` and `stopped`.
- **Bounded output.** Messages are written with `snprintf()` into buffers whose size is passed in.

5.2 System Workflow

A debugging session follows the same sequence every time, whichever front end is used. The steps below describe it from start to finish.

Session workflow

1. The user starts `my_debugger <program>` (or enters the program path in the GUI and clicks RUN).
2. The debugger calls `debugger_init()` and then `debugger_launch()`. A child process is created with `fork()`; the child enables tracing with `PTRACE_TRACEME` and runs the target with `execl()`.
3. The target stops at its first instruction after exec. The debugger waits with `waitpid()`, enables `PTRACE_O_TRACESYSGOOD`, and reports the PID.
4. The user issues commands: set a breakpoint, continue, step, view registers, read memory, trace system calls, or stop.
5. For each command the debugger performs the required `ptrace` operations, calls `waitpid()`, and decodes the new state of the target.
6. A text message describing the result is shown to the user.
7. When the target exits or is killed, or the user quits, the session ends.

What each command does internally

Command	Internal sequence
Set breakpoint	Check state → PEEKDATA original word → save it → patch low byte with 0xCC → POKEDATA
Continue	If stopped at breakpoint: restore byte, RIP = bp, SINGLESTEP, re-install → PTRACE_CONT → waitpid → decode status
Step	Same step-over preparation if needed → PTRACE_SINGLESTEP → waitpid → re-install breakpoint if it was removed
Registers	Check state → PTRACE_GETREGS → format 18 values
Memory	Check state → clear errno → PTRACE_PEEKDATA → check errno → format address and value
System-call trace	Loop 10 times: PTRACE_SYSCALL → waitpid → GETREGS → print ENTER or EXIT
Stop	kill(pid, SIGKILL) → waitpid → clear flags

Behaviour after the target ends

When `waitpid()` reports that the target exited or was killed, the core sets `running = 0` and `stopped = 0`. The command-line front end then leaves its prompt loop, because its loop

condition is `while (dbg.running)`. The GUI shows "Status: FINISHED". Any later command returns the message "No running program."

5.3 Flowchart / Data Flow

Figure 5.2 shows the flow of control of a complete debugging session. It starts with launching the target and then repeats a command loop until the program ends.

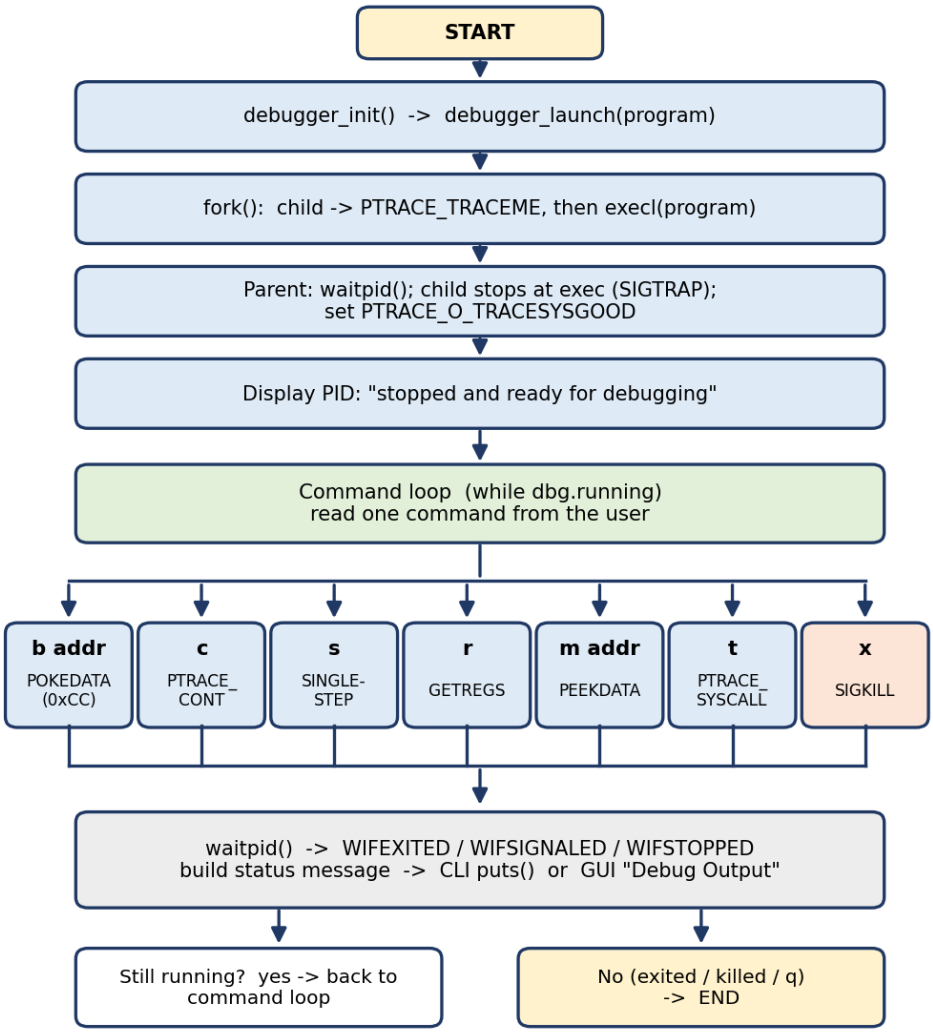


Figure 5.2: Flowchart of a debugging session

Reading the flowchart

After the launch sequence the debugger enters the command loop. Each of the seven commands in the second row performs its own `ptrace` operation. All of them end the same way: `waitpid()` returns a status, the status is decoded as exited, signalled or stopped, a text message is built, and the front end displays it. If the target is still alive the loop continues; otherwise the session ends.

Data flow

Data item	From	To	Form
Command or button click	User	Front end	Text / event

Data item	From	To	Form
Parsed address	Front end	Core function	`unsigned long long` (parsed with `strtoull`)
`Debugger *dbg`	Front end	Core function	Pointer to the session state
ptrace requests	Core	Kernel	System-call arguments
Wait status, registers, memory words	Kernel	Core	`int` status, `struct user_regs_struct`, `long`
Result message	Core	Front end	`char message[]` buffer
Displayed output	Front end	User	Terminal text or Debug Output pane

The single "message buffer" path keeps the core independent of the user interface: the core never prints anything itself.

Error flow

Errors follow the same path as normal results. When a check or a system call fails, the core writes an explanation into the message buffer and returns -1. The front end displays the message exactly as it would display a success message, and the GUI sets the status to ERROR or TRACE ERROR where appropriate. The session is never terminated by an error in a command, so the user can correct the mistake and continue.

5.4 Module Description

The debugger is divided into modules, each implemented by one public function plus a few private helpers. The table summarises them.

Module summary

Module	Function	Description
Initialisation	`debugger_init()`	Clears the `Debugger` structure and sets default values (`pid = -1`)
Launch	`debugger_launch()`	fork + PTRACE_TRACEME + execl; waits for first stop; enables TRACESYSGOOD
Breakpoint	`debugger_set_breakpoint()`	Saves the original word and writes the INT3 byte (0xCC) at the address
Continue	`debugger_continue()`	Steps over an active breakpoint if needed, then resumes with PTRACE_CONT and reports the stop reason
Step	`debugger_step()`	Executes exactly one instruction with PTRACE_SINGLESTEP
Registers	`debugger_get_registers()`	Reads and prints RAX–R15, RIP and EFLAGS with PTRACE_GETREGS
Memory	`debugger_read_memory()`	Reads one 8-byte word from a given address with PTRACE_PEEKDATA
System-call trace	`debugger_trace_syscalls()`	Uses PTRACE_SYSCALL to show ENTER / EXIT of the next 10 system calls with names
Stop	`debugger_stop()`	Kills the target with SIGKILL and collects it with waitpid
CLI front end	`main.c`	Text prompt "(debugger)" with single-letter commands
GUI front end	`gui.c`	GTK+ window with entries, buttons, status label and Debug Output pane

Private helper functions in debugger.c

Helper	Purpose
`msg()`	Copies a fixed text into the caller's message buffer safely
`wait_for_child()`	Wraps `waitpid()` for the target process
`read_word()`	Reads one word with PTRACE_PEEKDATA, using `errno` to detect failure
`write_word()`	Writes one word with PTRACE_POKEDATA
`restore_breakpoint_instruction()`	Writes the saved original word back to the breakpoint address
`syscall_name()`	Converts a system-call number into a name

Preconditions and error messages

Each public function checks its preconditions before acting and reports a specific message if one

is not met.

Module	Precondition	Message when violated
Breakpoint	Running, stopped, no existing breakpoint	"No running program." / "Program must be stopped before setting a breakpoint." / "This demo supports one breakpoint at a time."
Continue	Running and stopped	"No running program." / "Program is already running."
Step	Running and stopped	"No running program." / "Program must be stopped before single stepping."
Registers, memory	Running and stopped	"No running program." / "Program must be stopped."
System-call trace	Running and stopped	"No running program." / "Program must be stopped before tracing."

5.5 Input and Output Design

The debugger accepts input in two ways, a command line and a graphical window, and both present results as plain text messages.

Command-line input

Command	Action
`b <address>`	Set a breakpoint (e.g. `b 0x401136`)
`c`	Continue execution
`s`	Execute a single instruction
`r`	Show registers
`m <address>`	Read memory at the address (e.g. `m 0x401136`)
`t`	Trace the next 10 system calls
`x`	Stop (kill) the target program
`h`	Help
`q`	Quit the debugger

The command is chosen by its first character. Addresses are converted with `strtoull(..., 0)`, so both decimal and `0x` hexadecimal notation are accepted. If no number follows `b` or `m`, the message "Usage: b <address>" or "Usage: m <address>" is shown.

Graphical input

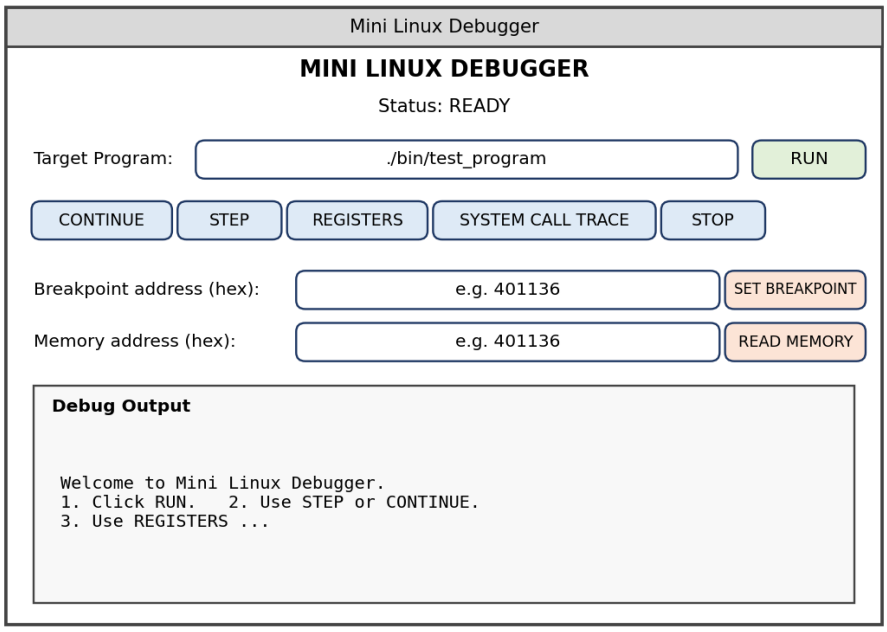


Figure 5.3: Layout of the GTK+ 3 window (schematic)

Control	Purpose	Validation
Target Program entry	Path of the program to debug (default `./bin/test program`)	Must not be empty

Control	Purpose	Validation
RUN	Launch the target	Message shown if the path is empty
CONTINUE, STEP, REGISTERS, STOP	Run-control and inspection	State checked in the core
SYSTEM CALL TRACE	Trace 10 system calls	GUI checks "No running program" and "must be stopped"
Breakpoint address (hex) + SET BREAKPOINT	Set the breakpoint	`strtoull(text, &end, 16)`; must be a complete hex number
Memory address (hex) + READ MEMORY	Read a word	Same hex validation

Output design

- **Debug Output pane / terminal:** shows launch information, breakpoint messages, the register table, the memory value and the system-call trace.
- **Status label (GUI):** shows the current state: READY, STOPPED, BREAKPOINT SET, SYSCALL TRACE PAUSED, FINISHED, ERROR or TRACE ERROR.
- **Error messages:** clear sentences such as "No running program." or "Failed to read memory at 0x0."

Format of the main outputs

```
REGISTERS
------------------------------
RAX = 0xa
RBX = 0x7ffc...
...
RIP = 0x401137
EFLAGS = 0x206

MEMORY
------------------------------
Address: 0x401136
Value:   0xe5894855fa1e0fcc
```

CHAPTER 6 IMPLEMENTATION

6.1 Module-wise Implementation

This section describes how each module of the debugger is implemented. All modules except the front ends are in `src/debugger.c` (1085 lines). Every public function follows the same pattern: check the state, perform `ptrace` operations, call `waitpid()` where needed, and write a message for the user.

Launch module – debugger_launch()

`debugger_launch()` calls `fork()`. In the child, `ptrace(PTRACE_TRACEME)` is called, followed by `execl(program, program, NULL)`. In the parent, `waitpid()` returns when the child stops after the exec. If the child had already exited (for example because the program name was wrong) the function reports "Program exited immediately with status 1." Otherwise it calls `PTRACE_SETOPTIONS` with `PTRACE_O_TRACESYSGOOD` and prints the PID. It sets `running = 1` and `stopped = 1`.

Breakpoint module – debugger_set_breakpoint()

The function first checks that the program is running and stopped and that no other breakpoint is set. It reads the 8-byte word at the address with `PTRACE_PEEKDATA`, stores it in `saved_word`, replaces the lowest byte with `0xCC`, and writes the word back with `PTRACE_POKEDATA`. Figure 6.1 shows the bytes at the start of `calculate()` before and after the change.

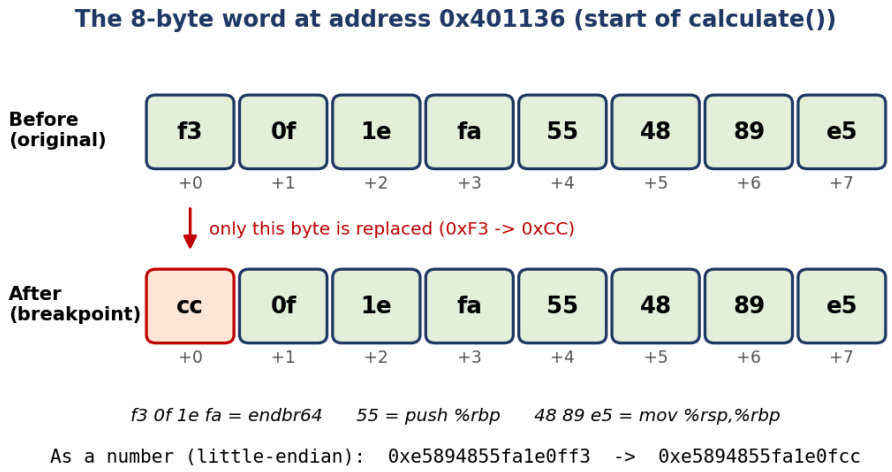


Figure 6.1: The breakpoint changes one byte of the word at 0x401136

Continue and Step modules

When the CPU executes INT3, the kernel stops the program with SIGTRAP and RIP points **one byte after** the breakpoint address, because INT3 is a one-byte instruction. Before resuming, `debugger_continue()` and `debugger_step()` therefore check whether `RIP ==`

`breakpoint_addr + 1`. If so, they restore the original byte, set RIP back to `breakpoint_addr`, and single-step the original instruction. `debugger_continue()` then re-installs the breakpoint and resumes with `PTRACE_CONT`; `debugger_step()` re-installs it after its own single step.

Register and Memory modules

`debugger_get_registers()` uses `PTRACE_GETREGS` to fill a `struct user_regs_struct` and formats 18 values in hexadecimal: RAX, RBX, RCX, RDX, RSI, RDI, RBP, RSP, R8–R15, RIP and EFLAGS. `debugger_read_memory()` uses `PTRACE_PEEKDATA`. Because the call can legitimately return -1 as data, success is judged by `errno`, which is cleared before the call.

System-call trace module

`debugger_trace_syscalls()` loops until ten calls have been traced. In each round it calls `PTRACE_SYSCALL`, waits, and checks whether the stop signal equals `SIGTRAP | 0x80`. The first such stop of a pair is the **entry**: the number is read from `orig_rax`. The second is the **exit**: the return value is read from `rax`. The function `syscall_name()` turns common numbers into names. If the program exits during tracing, the function reports its exit status.

Stop module – debugger_stop()

`debugger_stop()` sends `SIGKILL` with `kill()`, collects the exit status with `waitpid()` so that no zombie process is left, and clears the `running`, `stopped` and `breakpoint_set` flags.

6.2 Algorithm / Pseudocode

The algorithms below describe the main operations in pseudocode. They follow the implementation closely but leave out error handling for clarity.

Algorithm 1 – Launch

```
pid = fork()
if pid == 0:                        # child
    ptrace(TRACEME)
    execl(program)
else:                               # parent
    waitpid(pid)                    # child stops at exec
    if child already exited: report and return
    ptrace(SETOPTIONS, TRACESYSGOOD)
    running = 1 ; stopped = 1
```

Algorithm 2 – Set breakpoint

```
check running, stopped, no existing breakpoint
original = PEEKDATA(addr)
saved_word = original
patched = (original & ~0xFF) | 0xCC
POKEDATA(addr, patched)
breakpoint_addr = addr ; breakpoint_set = 1
```

Algorithm 3 – Continue (with step-over)

```
if breakpoint_set and RIP == breakpoint_addr + 1:
    POKEDATA(breakpoint_addr, saved_word)     # restore original
    RIP = breakpoint_addr ; SETREGS
    SINGLESTEP ; waitpid                      # run original instruction
    POKEDATA(breakpoint_addr, saved | 0xCC)   # put breakpoint back
PTRACE_CONT ; waitpid
decode status:
    exited / killed  -> running = 0
    SIGTRAP at bp+1  -> "Breakpoint hit"
    other signal     -> report signal number
```

Algorithm 4 – Single step

```
if breakpoint_set and RIP == breakpoint_addr + 1:
    restore original byte ; RIP = breakpoint_addr ; SETREGS
    SINGLESTEP ; waitpid
    re-install 0xCC
else:
    SINGLESTEP ; waitpid
report "One instruction executed" (or exit / signal)
```

Algorithm 5 – System-call trace

```
count = 0 ; syscall_entry = 1
while count < 10:
    PTRACE_SYSCALL ; waitpid
    if exited or killed: finish
    if stop signal == (SIGTRAP | 0x80):
        GETREGS
        if syscall_entry: print ENTER, orig_rax, name ; syscall_entry = 0
        else:             print EXIT, rax ; syscall_entry = 1 ; count++
    else: report signal and stop tracing
```

Complexity

Operation	ptrace calls	Notes
Launch	1 (TRACEME) + 1 (SETOPTIONS)	Plus one `fork` and one `exec`
Set breakpoint	2 (PEEK, POKE)	Constant time
Continue from a breakpoint	Up to 5 (GETREGS, POKE, SETREGS, SINGLESTEP, POKE) + 1 (CONT)	Constant overhead for the step-over
Registers / memory	1	Constant time
System-call trace	2 per call (entry + exit) plus GETREGS each	10 calls = 20 stops

6.3 Core C / C++ Implementation

The core of the project is `debugger.c`. This section shows the most important parts of the real source code and explains them. The code is written in C; no C++ is used.

Debugger state (debugger.h)

The state of a session is a single structure. It is passed by pointer to every function.

```
typedef struct {
    pid_t pid;
    int running;
    int stopped;
    int breakpoint_set;
    unsigned long long breakpoint_addr;
    unsigned long long saved_word;
    /* System-call tracing */
    int syscall_tracing;
    int syscall_entry;
} Debugger;
```

Starting the target (child side)

The child enables tracing and then replaces itself with the target. If either step fails, it prints the reason and ends with status 1, which the parent reports as "Program exited immediately with status 1."

```
    if (pid == 0) {
        /* Child process */
        if (ptrace(PTRACE_TRACEME, 0, NULL, NULL) == -1) {
            perror("PTRACE_TRACEME");
            _exit(1);
        }
        execl(program, program, NULL);
        perror("execl");
        _exit(1);
    }
```

Reading and writing target memory

Both helpers use a single `ptrace` request. `read_word` clears `errno` first because a valid word can also be -1.

```
static long read_word(Debugger *dbg, unsigned long long addr)
{
    errno = 0;
    long value = ptrace(
        PTRACE_PEEKDATA,
        dbg->pid,
        (void *)addr,
        NULL
    );
    return value;
}
```

```
static int write_word(Debugger *dbg,
                      unsigned long long addr,
                      unsigned long long value)
{
    if (ptrace(
            PTRACE_POKEDATA,
            dbg->pid,
            (void *)addr,
            (void *)value) == -1) {
        return -1;
    }
```

Inserting the INT3 breakpoint

Only the lowest byte of the word is replaced; the other seven bytes are kept.

```
    unsigned long long patched =
        ((unsigned long long)original & ~0xFFULL) | 0xCCULL;
```

Recognising a breakpoint stop

After `PTRACE_CONT` returns, a SIGTRAP with RIP one byte past the breakpoint address means the breakpoint was hit.

```
        if (signal == SIGTRAP) {
            struct user_regs_struct regs;
            if (ptrace(PTRACE_GETREGS,
                       dbg->pid,
                       NULL,
                       &regs) == 0) {
                if (dbg->breakpoint_set &&
                    regs.rip == dbg->breakpoint_addr + 1) {
                    snprintf(message,
                             message_size,
                             "Breakpoint hit at 0x%llx.\n"
                             "Program is stopped.",
                             dbg->breakpoint_addr);
                    return 0;
                }
```

Recognising a system-call stop

With `PTRACE_O_TRACESYSGOOD`, a system-call stop carries the signal number `SIGTRAP | 0x80`. The `syscall_entry` flag alternates between entry and exit.

```
        if (signal == (SIGTRAP | 0x80)) {
            struct user_regs_struct regs;
            if (ptrace(PTRACE_GETREGS,
                       dbg->pid,
                       NULL,
                       &regs) == -1) {
                dbg->syscall_tracing = 0;
                msg(message, message_size,
                    "PTRACE_GETREGS failed during syscall trace.");
                return -1;
            }
```

```
            if (dbg->syscall_entry) {
                /*
                 * orig_rax contains the system-call number
                 * on x86-64 Linux.
                 */
                long syscall_number =
                    (long)regs.orig_rax;
                used += snprintf(
                    message + used,
                    message_size - used,
                    "[%d] ENTER  syscall=%ld (%s)\n",
                    count + 1,
                    syscall_number,
                    syscall_name(syscall_number)
                );
                dbg->syscall_entry = 0;
            } else {
                /*
                 * rax contains the return value when
                 * the system call finishes.
                 */
                long return_value =
                    (long)regs.rax;
                used += snprintf(
                    message + used,
                    message_size - used,
                    "[%d] EXIT   return=%ld\n",
                    count + 1,
                    return_value
                );
                dbg->syscall_entry = 1;
                count++;
            }
```

6.4 Linux System Calls / APIs Used

Every effect that the debugger has on the target is produced by one of the system calls or `ptrace` requests below.

System calls and ptrace requests

API / request	Purpose	Used in
`fork()`	Create the child process	debugger_launch
`execl()`	Run the target program in the child	debugger_launch
`PTRACE_TRACEME`	Child allows its parent to trace it	debugger_launch
`PTRACE_SETOPTIONS`	Enable `PTRACE O TRACESYSGOOD`	debugger_launch
`PTRACE_PEEKDATA`	Read a word from target memory	set_breakpoint, read_memory
`PTRACE_POKEDATA`	Write a word to target memory	set_breakpoint, continue, step
`PTRACE GETREGS / SETREGS`	Read / write CPU registers	registers, continue, step, trace
`PTRACE_CONT`	Resume the target	debugger_continue
`PTRACE_SINGLESTEP`	Execute one instruction	debugger_step, continue
`PTRACE_SYSCALL`	Resume until next system-call entry or exit	debugger_trace_syscalls
`waitpid()`	Wait for the target to change state	all run-control functions
`kill(pid, SIGKILL)`	Terminate the target	debugger_stop
`strtoull()`	Convert a typed address	main.c, gui.c

Decoding the wait status

Macro	Meaning	Action taken
`WIFEXITED(status)`	Target ended normally	`running = 0`; report "exited normally with status N"
`WIFSIGNALED(status)`	Target was killed by a signal	`running = 0`; report "terminated by signal N"
`WIFSTOPPED(status)`	Target stopped	`stopped = 1`; examine `WSTOPSIG(status)`
`WSTOPSIG == SIGTRAP`	Breakpoint or single step	Compare RIP with `breakpoint_addr + 1`
`WSTOPSIG == SIGTRAP | 0x80`	System-call stop	Print ENTER or EXIT

System-call names

System-call names are resolved through `syscall_name()`, which recognises 24 common x86-64 calls (listed in Section 4.3). Any other number is shown as "unknown". In the sample trace, numbers 21 (`access`) and 17 (`pread64`) appear as "unknown" because they are not yet in the table; adding them takes one line each.

Error handling for each call

Call	Failure condition	Message produced
`fork()`	Returns -1	"fork() failed."
`ptrace(PTRACE_TRACEME)` (child)	Returns -1	`perror("PTRACE_TRACEME")`, child ends with status 1
`execl()` (child)	Returns (only on error)	`perror("execl")`; parent reports "Program exited immediately with status 1."
`waitpid()`	Returns -1	"waitpid() failed."
`ptrace(PTRACE_SETOPTIONS)`	Returns -1	"PTRACE_SETOPTIONS failed."
`PTRACE_PEEKDATA`	`errno` set	"Failed to read memory at 0x…"
`PTRACE_POKEDATA`	Returns -1	"Failed to write breakpoint."
`PTRACE_CONT` / `SINGLESTEP` / `SYSCALL`	Returns -1	"PTRACE_CONT failed." etc.
`PTRACE_GETREGS`	Returns -1	"PTRACE_GETREGS failed."
`kill()`	Returns -1	"Failed to stop program."

The system calls behind the library wrappers

The functions used in the code are wrappers in the C library. On x86-64 Linux they end in the following system-call numbers, which are the same kind of numbers that the trace reports for the target.

Wrapper	System call	Number
`waitpid()`	`wait4`	61
`kill()`	`kill`	62
`ptrace()`	`ptrace`	101
`execl()`	`execve`	59

`fork()` is implemented by the C library with the `clone` system call. Since the debugger's own system calls are not shown by its trace (which traces the target), their numbers can be confirmed by running `strace ./bin/my_debugger ./bin/test_program`.

6.5 Process / Thread Implementation

The project uses a **two-process model**. The debugger is the parent process; the program being debugged is its child. The debugger is single-threaded, and so is the sample target.

Process creation

`fork()` returns twice: it returns 0 in the child and the child's PID in the parent. The code uses this to separate the two roles. The child never returns from `debugger_launch()`: it either becomes the target through `execl()` or ends with `_exit(1)`. The PID returned to the parent is stored in `dbg->pid` and used in every later `ptrace` and `waitpid` call.

States of the target process

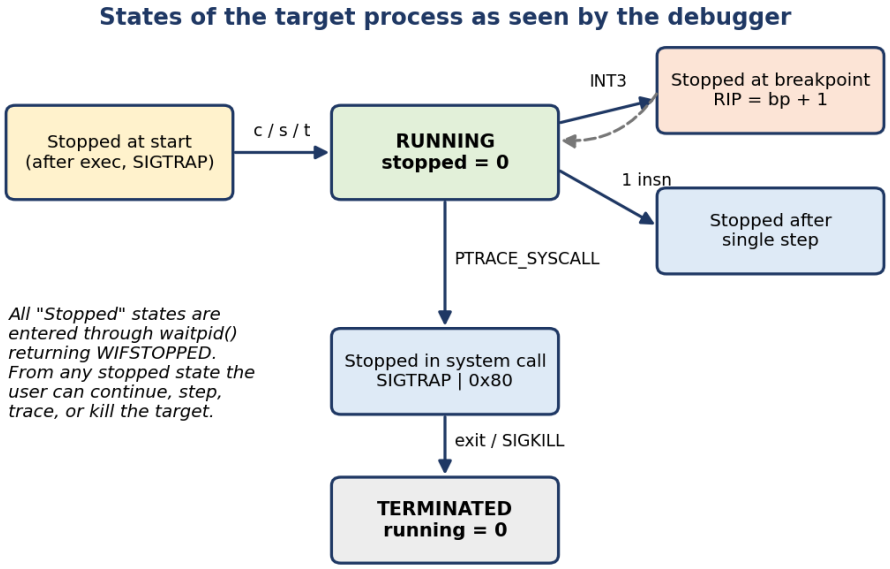


Figure 6.2: States of the target as seen by the debugger

State of target	How it is entered	How the debugger sees it
Stopped at start	After exec with TRACEME	waitpid returns WIFSTOPPED (SIGTRAP)
Running	After CONT / SINGLESTEP / SYSCALL	`stopped = 0` in the Debugger structure
Stopped at breakpoint	INT3 executed, SIGTRAP raised	RIP = breakpoint address + 1
Stopped in system call	PTRACE_SYSCALL stop	Signal equals SIGTRAP | 0x80
Terminated	Normal exit or SIGKILL	WIFEXITED / WIFSIGNALED; `running = 0`

Process termination

A target can end in three ways: it finishes normally (`WIFEXITED`), it is killed by a signal (`WIFSIGNALED`), or the user stops it. In the last case `debugger_stop()` sends SIGKILL and then calls `waitpid()`, so the kernel can release the process's entry. The CLI also stops the target

when the user types `q` while it is still running.

Threads

No threads are created by the debugger. The target is a single-threaded program; debugging a multi-threaded target would require tracing every new thread (`PTRACE_O_TRACECLONE`), which is outside the scope of this project (see Section 4.4).

6.6 IPC / Synchronization Implementation

Communication between the debugger and its target is carried out entirely by kernel-mediated mechanisms. There is no shared memory, pipe or socket between the two processes.

Communication channels

Direction	Mechanism	Carries
Debugger → target	`ptrace` requests	Resume, step, read / write registers and memory
Target → debugger	Signals and the `waitpid()` status	SIGTRAP at breakpoints and steps, SIGTRAP | 0x80 at system calls, exit or kill status
Debugger → target	`kill(SIGKILL)`	Termination request
Debugger → user	Message buffer	Text result of each command

Ordering of events during CONTINUE

Step	Debugger	Kernel / target
1	Calls `ptrace(PTRACE_CONT)`	Target is resumed and runs
2	Calls `waitpid()` and blocks	Target executes until INT3 or exit
3	—	Target traps; kernel stops it and sends SIGTRAP to the tracer
4	`waitpid()` returns; status is decoded	Target remains stopped
5	Reads RIP with `PTRACE_GETREGS`; compares with `breakpoint addr + 1`	—
6	Prints "Breakpoint hit at 0x…" and returns	Target waits for the next request

How synchronization is guaranteed

- **Blocking wait.** After every resume request the debugger blocks in `waitpid()`, so it cannot read registers or memory while the target is running.
- **State flags.** `running`, `stopped` and `syscall_entry` in the `Debugger` structure make each command check that its action is legal at that moment.
- **Single tracer.** The target has exactly one tracer, the parent that created it; the kernel rejects requests from any other process.

Race conditions avoided

The most dangerous race in a debugger is changing a program's memory while it runs. Here this cannot happen: the breakpoint byte is written only while the target is stopped, and the breakpoint is re-installed only after the single step has completed and `waitpid()` has returned. Likewise, `debugger_stop()` always calls `waitpid()` after `kill()` so that the exit status is collected before the session is declared closed.

6.7 Memory Management Implementation

Two kinds of memory matter in this project: the memory of the target process, which the debugger reads and patches, and the debugger's own buffers. Each is handled carefully.

Accessing the target's memory

The debugger cannot use the target's addresses directly, because they belong to a different virtual address space. It asks the kernel to read or write one 8-byte word at a time with `PTRACE_PEEKDATA` and `PTRACE_POKEDATA`. The words are 64-bit values; on x86-64 the least significant byte is stored at the lowest address (little-endian). That is why the memory word `0xe5894855fa1e0fcc` read at `0x401136` ends with `cc`: the INT3 byte sits at the lowest address.

Breakpoint patching

- Only the lowest byte of the word is changed to `0xCC`; the other seven bytes are preserved.
- The original word is stored in `saved_word` and written back when the breakpoint is stepped over.
- The breakpoint is written only while the target is stopped.

Error detection

`PTRACE_PEEKDATA` returns the word itself, so -1 can be valid data. The code therefore sets `errno` to 0 before the call and tests `errno` afterwards. A failed read is reported as "Failed to read memory at 0x…", for example when address 0 is requested.

Where things live in the target

Region	Typical address	Evidence
Program code (non-PIE)	`0x401136` (calculate), `0x401157` (main)	objdump of `test_program`
Stack	`0x7ffc…` range	RSP and RBP values in the register display
Shared libraries and mmap area	`0x7f…` range	Return values of `mmap` in the system-call trace
Heap (brk)	Address returned by `brk`	Return value of `brk` in the trace

Stack, library and heap addresses change from run to run because of address-space randomisation. Only the program code of `test_program` is fixed, which is why it is built with `-no-pie`.

The debugger's own buffers

Buffer	Size	Protection

Buffer	Size	Protection
`char message[4096]` (CLI and GUI)	4096 bytes	Written only with `snprintf()`
`char message[8192]` (system-call trace in the GUI)	8192 bytes	Trace stops with "Output limit reached." when nearly full
`char command[100]` (CLI)	100 bytes	Read with `fgets()`, which limits the input length

6.8 Optional User Interface Implementation

The graphical interface is implemented in `gui/gui.c` (785 lines) with GTK+ 3. It is optional: the debugger works completely from the command line, and the GUI only gives the same commands a friendlier form.

Building the window

The function `activate()` creates the window with `GtkApplication` and then builds it from top to bottom: a title label, a status label, a row with the target-program entry and the RUN button, a row of control buttons, a row for the breakpoint address, a row for the memory address, and a scrollable read-only monospace `GtkTextView` called Debug Output. A welcome text with six numbered hints is placed in the output pane when the window opens.

Event handling

Each button is connected with `g_signal_connect()` to a callback function. A callback reads the entry fields if needed, calls the matching debugger function, appends the returned message to the output pane, and updates the status label.

Button	Callback	Status label set to
RUN	`on_launch`	STOPPED (or ERROR)
CONTINUE	`on_continue`	STOPPED, or FINISHED when the program ends
STEP	`on_step`	STOPPED, or FINISHED
REGISTERS	`on_registers`	(unchanged)
SYSTEM CALL TRACE	`on_trace_syscalls`	SYSCALL TRACE PAUSED, FINISHED or TRACE ERROR
STOP	`on_stop`	READY
SET BREAKPOINT	`on_set_breakpoint`	BREAKPOINT SET
READ MEMORY	`on_memory`	(unchanged)

Helper functions

Three small helpers keep the callbacks short: `append_text()` adds a message and a blank line at the end of the output, `set_status()` changes the status label, and `parse_address()` converts the typed hexadecimal text with `strtoull()` and rejects incomplete numbers.

```
static int parse_address(const char *text,
                         unsigned long long *value)
{
    char *end = NULL;
    errno = 0;
    unsigned long long v =
        strtoull(text, &end, 16);
    if (errno != 0 ||
        end == text ||
        *end != '\0') {
        return -1;
    }
    *value = v;
    return 0;
}
```

Shared design with the CLI

The GUI keeps one global `Debugger` object, just as the CLI keeps one local object, and both call identical core functions. The two front ends therefore behave in the same way and produce the same messages.

Limitations of the GUI

All debugger calls run on the GTK main thread, so the window cannot redraw while the debugger waits for the target. For the short sample program this is not noticeable; a long-running target would freeze the window until it stops.

CHAPTER 7 EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

This section explains how to build the project on Ubuntu. The whole project is built with one command, and the individual steps are shown so that each file can also be compiled by hand.

Project directory layout

```
project/
  Makefile
  include/debugger.h
  src/debugger.c        (debugger core)
  src/main.c            (command-line front end)
  gui/gui.c             (GTK+ 3 front end)
  test/test_program.c   (sample target program)
  bin/                  (output executables)
```

Building with make

```
make
```

This runs the targets `test_program`, `debugger` and `gui` in turn and produces `bin/test_program`, `bin/my_debugger` and `bin/debugger_gui`. For the first two, `make` prints the exact compiler commands it runs:

```
gcc -Wall -Wextra -g -O0 -Iinclude -no-pie test/test_program.c -o bin/test_program
gcc -Wall -Wextra -g -O0 -Iinclude src/debugger.c src/main.c -o bin/my_debugger
```

The two commands finish without any warning. The GUI is built with the third command, which adds the GTK+ flags:

```
gcc -Wall -Wextra -g -O0 -Iinclude src/debugger.c gui/gui.c -o bin/debugger_gui \
    $(pkg-config --cflags --libs gtk+-3.0)
```

Building only what is needed

Command	Builds
`make test_program`	Only the sample target
`make debugger`	Only the command-line debugger (no GTK needed)
`make gui`	Only the GTK+ debugger
`make clean`	Removes the three executables
`make run`	Builds everything and starts the GUI

Checking the result

The `file` command confirms that the target was built as a fixed-address executable, while the debugger is a normal position-independent executable:

```
$ file bin/test_program
bin/test_program: ELF 64-bit LSB executable, x86-64, ...
$ file bin/my_debugger
bin/my_debugger: ELF 64-bit LSB pie executable, x86-64, ...
```

Troubleshooting

Message	Cause	Remedy
`gtk/gtk.h: No such file or directory`	GTK development files missing	`sudo apt install libgtk-3-dev pkg-config`
`No such file or directory` for `../include/debugger.h`	Compiling from the wrong directory	Run `make` from the project root
`bin/...: No such file or directory`	`bin` directory missing	`mkdir -p bin`
`Package gtk+-3.0 was not found`	`pkg-config` cannot find GTK	Install `libgtk-3-dev`, or build only `make debugger`

7.2 Execution Commands

This section explains how to start the debugger, which commands are available, and how to carry out a typical session.

Starting the debugger

```
# Command-line debugger
./bin/my_debugger ./bin/test_program

# Graphical debugger
./bin/debugger_gui          (or: make run)
```

The command-line program expects exactly one argument, the program to debug. If it is missing, it prints "Usage: ./bin/my_debugger <program>". The GUI shows `./bin/test_program` in its Target Program field; click RUN to start it.

Finding the addresses to use

The sample target, `test_program.c`, adds two numbers in `calculate()`, doubles the result and prints it. Run normally it prints `Result = 60`. Using `objdump -d`, the addresses in the compiled target are:

Symbol	Address	Description
`calculate`	0x401136	First instruction of `calculate()`; used as the breakpoint in the demonstration
`main`	0x401157	Start of `main()`
`call calculate`	0x40117b	Instruction in `main()` that calls `calculate()`

Command reference

CLI command	GUI equivalent	Effect
(program argument)	Target Program + RUN	Launch the target; it stops before running
`b 0x401136`	Breakpoint field + SET BREAKPOINT	Install the breakpoint
`c`	CONTINUE	Run until the breakpoint, exit or a signal
`s`	STEP	Execute one instruction
`r`	REGISTERS	Show registers
`m 0x401136`	Memory field + READ MEMORY	Read one word at the address
`t`	SYSTEM CALL TRACE	Trace the next 10 system calls
`x`	STOP	Kill the target
`h`, `q`	—	Help; quit

A typical session

1. Build with `make` and find the address of `calculate()` with `objdump -d`.

2. Start the debugger and note the PID shown.
3. Set a breakpoint at `0x401136`.
4. Continue; the debugger reports "Breakpoint hit at 0x401136."
5. Show the registers; note RIP, RAX and RSP.
6. Step a few times, showing the registers after each step.
7. Read memory at the breakpoint address and observe the `cc` byte.
8. Continue to the end; the target prints `Result = 60` and exits.

7.3 Sample Inputs and Outputs

The outputs in this section were produced by running the debugger on `bin/test_program`. PIDs and stack addresses differ from run to run.

Session 1 – Breakpoint, registers, step and memory

Input commands: `b 0x401136`, `c`, `r`, `s`, `m 0x401136`, `c`. Registers RBX–R15 are omitted below for space.

```
Program launched.
PID: 139
Status: stopped and ready for debugging.

(debugger) Breakpoint set at 0x401136.
Original instruction byte saved.

(debugger) Breakpoint hit at 0x401136.
Program is stopped.

(debugger) REGISTERS
------------------------------
RAX = 0xa
...
RSP = 0x7ffc83fde0b8
RIP = 0x401137
EFLAGS = 0x206

(debugger) One instruction executed. Program is stopped.

(debugger) MEMORY
------------------------------
Address: 0x401136
Value:   0xe5894855fa1e0fcc

(debugger) Result = 60
Program exited normally with status 0.
```

Explanation. After the breakpoint is hit, RIP is `0x401137`, exactly one byte after the breakpoint address, because INT3 is a one-byte instruction. RAX is `0xa` (10), the first argument. The memory word at `0x401136` ends with the byte `cc`, which proves that the INT3 instruction is installed in the program's code. After the final CONTINUE the program completes and prints `Result = 60`.

Session 2 – Single-stepping with registers

Input commands: `b 0x401136`, `c`, then `s` followed by `r` three times. The table shows the registers that changed.

After	RIP	RSP	RBP	Instruction just executed
Breakpoint hit	0x401137	…be8	…c00	(INT3 trap)
Step 1	0x40113a	…be8	…c00	`endbr64` (original instruction at 0x401136)
Step 2	0x40113b	…be0	…c00	`push %rbp` – RSP decreases by 8

After	RIP	RSP	RBP	Instruction just executed
Step 3	0x40113e	…be0	…be0	`mov %rsp,%rbp` – RBP now equals RSP

Throughout the session RAX = 0xa, RDI = 0xa and RSI = 0x14, the values of `x` (10) and `y` (20) passed to `calculate()`. The steps show the standard function prologue: the old frame pointer is pushed and the new frame is set up.

Session 3 – System-call trace at program start

```
(debugger) SYSTEM-CALL TRACE
============================
[1] ENTER  syscall=12 (brk)
[1] EXIT   return=1054470144
[2] ENTER  syscall=9 (mmap)
[2] EXIT   return=139856481067008
[3] ENTER  syscall=21 (unknown)
[3] EXIT   return=-2
[4] ENTER  syscall=257 (openat)
[4] EXIT   return=3
[5] ENTER  syscall=5 (fstat)
[5] EXIT   return=0
[6] ENTER  syscall=9 (mmap)
[6] EXIT   return=139856481013760
[7] ENTER  syscall=3 (close)
[7] EXIT   return=0
[8] ENTER  syscall=257 (openat)
[8] EXIT   return=3
[9] ENTER  syscall=0 (read)
[9] EXIT   return=832
[10] ENTER  syscall=17 (unknown)
[10] EXIT   return=784

System-call tracing paused after 10 syscall(s).
```

Explanation. These ten calls are made by the dynamic loader while the program is starting up: it sets up the heap (`brk`), maps memory (`mmap`), looks for the library cache (`access`, number 21, returning -2 = file not found), and opens and maps the C library (`openat`, `fstat`, `read`, `pread64`, `mmap`, `close`). Numbers 21 and 17 appear as "unknown" because they are not in the `syscall_name()` table. Return values of `mmap` and `brk` are addresses and change from run to run.

Session 4 – System-call trace after the breakpoint

Input commands: `b 0x401136`, `c`, `t`. The trace now covers the end of the program.

```
(debugger) Result = 60
SYSTEM-CALL TRACE
==============================
[1] ENTER  syscall=5 (fstat)
[1] EXIT   return=0
[2] ENTER  syscall=318 (unknown)
[2] EXIT   return=8
[3] ENTER  syscall=12 (brk)
[3] EXIT   return=688746496
[4] ENTER  syscall=12 (brk)
[4] EXIT   return=688881664
[5] ENTER  syscall=1 (write)
[5] EXIT   return=12
[6] ENTER  syscall=231 (exit_group)
Program exited with status 0.
```

Explanation. Call 5 is the `write` that prints the result: it returns 12, the number of bytes in `Result = 60` plus the newline. Call 6, `exit_group`, ends the program, so it has an ENTER but no EXIT, and the debugger reports "Program exited with status 0." The text `Result = 60` appears first on the screen because the target writes it to the terminal directly while the debugger's own message is printed afterwards.

Session 5 – Error handling

```
(debugger) Breakpoint set at 0x401136.
(debugger) This demo supports one breakpoint at a time.
(debugger) Failed to read memory at 0x0.
(debugger) Usage: m <address>
(debugger) Usage: b <address>
(debugger) Unknown command.   (followed by the list of commands)
```

Test-case summary

No.	Input	Expected result	Result
1	Start on `./bin/test program`	PID shown, target stopped	Pass
2	`b 0x401136`	Breakpoint set	Pass
3	Second `b …`	"This demo supports one breakpoint at a time."	Pass
4	`c` after breakpoint	"Breakpoint hit at 0x401136."	Pass
5	`r` at breakpoint	RIP = 0x401137	Pass
6	`s` three times	RIP 0x40113a, 0x40113b, 0x40113e	Pass
7	`m 0x401136` with breakpoint set	Word ends with `cc`	Pass
8	`m 0x0`	"Failed to read memory at 0x0."	Pass
9	`m zz` or `b` without address	Usage message	Pass
10	Unknown letter	"Unknown command." plus help	Pass
11	`c` without breakpoint	`Result = 60`, exit status 0	Pass
12	`t`	10 ENTER / EXIT pairs	Pass
13	`x`	"Program stopped."	Pass
14	Start on a missing program	"Program exited immediately with status 1."	Pass
15	Start with no argument	"Usage: ./bin/my_debugger <program>"	Pass

7.4 Screenshots of Execution

This section holds screenshots of the debugger running. Each box below should be replaced with a screenshot of your own run, taken with the commands given in the table.

Screenshots to include

Figure	What to capture	How to produce it
7.1	Command-line debugger after launch	`./bin/my_debugger ./bin/test_program`
7.2	Breakpoint set and hit	`b 0x401136` then `c`
7.3	Register display at the breakpoint	`r`
7.4	Single steps and memory read	`s`, `r`, then `m 0x401136`
7.5	System-call trace	`t`
7.6	GUI main window after RUN	`./bin/debugger gui`, click RUN
7.7	GUI with breakpoint and registers	Set breakpoint, CONTINUE, REGISTERS
7.8	GUI system-call trace	Click SYSTEM CALL TRACE

[Paste screenshot here]

Figure 7.1: Command-line debugger after launch

[Paste screenshot here]

Figure 7.2: Breakpoint set and hit (command line)

[Paste screenshot here]

Figure 7.3: Register display at the breakpoint

[Paste screenshot here]

Figure 7.4: Single steps and memory read

[Paste screenshot here]

Figure 7.5: System-call trace (command line)

[Paste screenshot here]

Figure 7.6: GUI main window after RUN

[Paste screenshot here]

Figure 7.7: GUI with breakpoint hit and registers

[Paste screenshot here]

Figure 7.8: GUI system-call trace

What the screenshots should show

- In the launch screenshot, the PID line and the "stopped and ready for debugging" status.
- In the breakpoint screenshots, the address `0x401136` in both the "Breakpoint set" and "Breakpoint hit" messages.
- In the register screenshot, RIP = 0x401137 and RAX = 0xa.
- In the trace screenshots, matching ENTER and EXIT lines with system-call names.

7.5 Demonstration of OS Concepts

The sessions in Section 7.3 show each major operating-system concept at work. This section links every concept to the evidence in the output, and then explains how the evidence can be double-checked.

Concepts and evidence

OS concept	Where it is demonstrated	Evidence in the output
Process creation	fork() + execl() at launch	A new PID is printed (e.g. PID: 139)
Process states	Stopped / running / terminated	"stopped and ready for debugging", later "exited normally"
Signals	SIGTRAP on breakpoint and single step; SIGKILL on stop	"Breakpoint hit at 0x401136"; "Program stopped."
CPU context	PTRACE_GETREGS	RIP = 0x401137, RAX = 0xa, RSP and EFLAGS displayed
Stack frames	Single steps through the function prologue	RSP falls by 8 after `push %rbp`; RBP = RSP after `mov %rsp,%rbp`
Virtual memory	PTRACE_PEEKDATA / POKEDATA	Memory word ends with `cc` after breakpoint insertion
System calls, user / kernel mode	PTRACE_SYSCALL trace	ENTER / EXIT pairs for brk, mmap, openat, read, write
Process synchronization	waitpid() after every resume	Output always appears only after the target has stopped
Process termination	Exit status and kill()	"Program exited normally with status 0."

Explaining the evidence

Process creation. The PID printed at launch belongs to a new process created by `fork()`. In another terminal, `ps` shows it as a child of the debugger while it is stopped.

Signals and traps. The reported breakpoint address is the address where INT3 was written, but RIP is one higher. This difference is the visible effect of the one-byte trap instruction and the reason for the "breakpoint address + 1" test in the code.

Kernel boundary. The trace shows that even a simple program makes many system calls before `main()` runs, and that `write` returning 12 corresponds to the 12 characters printed. This links the idea of "asking the kernel for a service" to a number that can be checked.

Cross-checking with standard tools

Check	Command	Expected agreement
System-call sequence	`strace ./bin/test_program`	Same sequence of calls as the trace (e.g. `write(1, "Result = 60\n", 12)`)
Function address	`objdump -d bin/test_program`	`calculate` at 0x401136

Check	Command	Expected agreement
Registers at the breakpoint	`gdb` with `break *0x401136`, `run`, `info registers`	RAX = 0xa, RDI = 0xa, RSI = 0x14
Process relationship	`ps -o pid,ppid,stat,cmd` during a stop	Target is the debugger's child, state `t` (traced stop)

CHAPTER 8 CONCLUSION

8.1 Summary of the Project

This project designed and implemented a Mini Linux Debugger in C using the Linux `ptrace()` interface. The debugger launches a target program under its control, inserts a software breakpoint using the INT3 instruction, continues or single-steps the program, displays CPU registers, reads process memory, and traces system calls. The debugging engine is kept separate from the user interface, so the same code is used by a command-line program and a GTK+ 3 graphical application.

Work carried out

Chapter	Content
1 Introduction	Background, problem, scope, objectives, importance and users
2 OS concepts	Process creation, ptrace, signals, registers, memory and system calls; existing tools; the proposed approach
3 Requirements	Hardware, software, Ubuntu / WSL environment and tools
4 Technology	C, GCC, system calls, synchronization, commands, GTK+
5 Design	Architecture, workflow, flowchart, modules, input and output
6 Implementation	Module code, algorithms, system calls, processes, IPC, memory and GUI
7 Demonstration	Compilation, execution, outputs, test cases and OS concepts shown

Overall outcome

The project shows in a practical way how an operating system supports debugging through process control, signals, system calls and virtual memory, and how a professional debugger such as GDB is built on the same foundations. The final program is small enough to read in full, yet complete enough to debug a real program from start to finish.

Limitations and future scope

Current limitation	Possible improvement
One breakpoint at a time	Keep a table of breakpoints, each with its own saved word
Reads one 8-byte word	Add a memory dump command and a memory write command
24 system-call names	Complete the table (for example `access`, `pread64`, `getrandom`)
No symbols or source lines	Look up function names with the ELF symbol table; later DWARF line tables
x86-64 Linux only	Abstract registers and the breakpoint instruction per architecture
Debugger runs on the GUI thread	Run debugger calls in a worker thread and update the window with `g idle add()`
No disassembly	Show the instruction at RIP using a disassembly library

Development process

The project was developed in stages, each of which could be tested before the next was started. The staged approach is typical for systems programs, because errors at a low level are hard to find once other features are built on them.

Stage	Work	Check before moving on
1	Launch the target under `PTRACE_TRACEME`	Program stops at start and PID is shown
2	Continue and exit handling	Target runs to completion and exit status is shown
3	Single-step and registers	RIP changes after each step
4	Memory read and breakpoint	Memory word shows `cc`; "Breakpoint hit" appears
5	Step-over of breakpoint	Program completes normally after a breakpoint
6	System-call trace	Paired ENTER / EXIT lines
7	Command-line front end, then GTK front end	Both give the same messages

Future roadmap

The limitations listed above can be removed in a natural order. Multiple breakpoints and a memory dump command are the most useful and the easiest. A completed system-call table is a very small change. Symbol lookup and disassembly need additional libraries and would form a larger follow-up project. Finally, moving the debugger calls to a worker thread would improve the GUI without changing the debugging logic.

8.2 Objectives Achieved

The objectives set in Section 1.4 have all been met. The table below states each objective, whether it was achieved, and the evidence in this report.

Achievement of objectives

Objective	Status	Evidence
Launch a program under ptrace control using fork / exec	Achieved	PID and "stopped and ready" message (7.3)
Breakpoint using INT3 with safe restoration	Achieved	RIP = bp + 1; word ends with `cc`; program still prints `Result = 60`
Continue and single-step execution	Achieved	RIP moves through 0x40113a, 0x40113b, 0x40113e
Read CPU registers and process memory	Achieved	18 registers and one-word memory read
Trace system calls with entry and exit	Achieved	Paired ENTER / EXIT lines in Sessions 3 and 4
Command-line and GUI front ends over one core	Achieved	`my_debugger` and `debugger_gui` share `debugger.c`

Discussion of the main achievements

Correct breakpoint handling. The most delicate part of the project is stepping over a breakpoint. The sessions show that after the breakpoint is hit, CONTINUE restores the byte, repositions RIP, executes the original instruction, re-installs the breakpoint, and resumes, so the target completes normally and prints its result.

Safe behaviour. Wrong usage never crashed the debugger. A second breakpoint, an unreadable address, an incomplete command, an unknown command, a missing program and a missing argument all produced clear messages (Section 7.3, test cases 3, 8, 9, 10, 14 and 15).

Reusable design. The core library has no user-interface code, which made it possible to add the GUI without changing the debugging logic.

Verification of each objective by test case

Objective	Test cases (Section 7.3)
Launch under control	1, 14, 15
Breakpoint with safe restoration	2, 3, 4, 7, 11
Continue and single-step	4, 6, 11
Registers and memory	5, 7, 8, 9
System-call trace	12
Safe handling of wrong input	3, 8, 9, 10, 14, 15

Quality attributes achieved

Attribute	Achievement
Correctness	Outputs agree with the program code and with expected register values
Robustness	No crash on any of the faulty inputs tried
Readability	Short functions, comments at each step, consistent style
Reusability	One core shared by two front ends
Portability	Runs on Ubuntu, WSL2 and similar x86-64 Linux systems

Objective not fully achieved

One goal was met only in part: the system-call trace shows a name for 24 call numbers, and a few common ones appear as "unknown". The objective of tracing system calls with entry and exit was achieved completely, since the numbers and return values are always shown; only the naming is incomplete.

8.3 Results Achieved

This section summarises what the working debugger produced when it was run on the sample program.

Observed results

- The debugger stopped the sample program exactly at `calculate()` (0x401136), showed RIP = 0x401137 and the INT3 byte `cc` in memory, then resumed correctly so that the program still printed `Result = 60`.
- Registers matched the values expected from the program code: RAX = 0xa, RDI = 0xa, RSI = 0x14 for `calculate(10, 20)`.
- Single-stepping through the function prologue changed RSP, RBP and RIP exactly as the instructions require.
- The system-call trace showed paired ENTER and EXIT events, including the `write` of 12 bytes and the final `exit_group`.
- All 15 test cases in Section 7.3 gave the expected result.

Results by feature

Feature	Result
Launch	Works; target stopped at start; errors for missing program handled
Breakpoint	Works at 0x401136; limited to one breakpoint
Continue / step	Work, including step-over of the breakpoint
Registers	All 18 values displayed
Memory	One word per request; invalid address reported
System-call trace	10 calls per request; two numbers (17, 21) and one more (318) shown as "unknown"
Stop	Target killed and collected
CLI and GUI	Both built from the same core

Observations and weaknesses

Testing showed that the system-call trace reports a few common calls as "unknown": 17 (`pread64`), 21 (`access`) and 318 (`getrandom`). This does not affect correctness, because the numbers are still shown, and it can be fixed by adding names to `syscall_name()`. Breakpoint addresses must be typed by hand, and only the sample target, built with `-no-pie`, has stable addresses. These weaknesses are listed with their remedies in Section 8.1.

Resource use and speed

The debugger is light. The command-line program has about 15 KB of code, uses fixed buffers

of 4 KB or 8 KB, and creates one child process. All commands on the sample program respond immediately, because each needs only a handful of system calls (see Section 6.2). The only noticeable delay would come from the target itself.

Measure	Observation
Size of `my_debugger` code	about 15 KB (`size` output)
Message buffers	4096 bytes (8192 for the GUI trace)
System calls for a breakpoint	2 (PEEK and POKE)
System calls to continue from a breakpoint	at most 6 `ptrace` calls plus one wait
Child processes created	1

Comparison with expectations

Expectation	Result
Program continues normally after a breakpoint	Yes – `Result = 60` printed and exit status 0
Reported breakpoint address equals installed address	Yes – 0x401136 in both messages
RIP one past the breakpoint when hit	Yes – 0x401137
Argument registers hold 10 and 20	Yes – RDI = 0xa, RSI = 0x14
`write` of the result returns its length	Yes – return value 12

8.4 Skills and Learning Outcomes

The project gave practical experience in operating-system programming, tool building and software engineering. The table links each skill to the activity in which it was learnt.

Technical skills

Skill	Learnt through
Linux process creation and states	Writing `debugger_launch()` with `fork` and `execl`
Using `ptrace()`, `waitpid()` and signals	Implementing every run-control command
x86-64 registers and the INT3 instruction	Breakpoint insertion and step-over logic
Reading and writing another process's memory	PEEKDATA / POKEDATA and `errno` handling
System-call numbering and the kernel boundary	The system-call trace
Systems programming in C	Structures, pointers, bit operations, safe string formatting
GUI programming with GTK+ 3	Building `gui.c` and connecting callbacks
Building and tooling	GCC flags, Makefile, `objdump`, `gdb`, `strace`

Software-engineering skills

- **Modular design:** separating a reusable core from two user interfaces.
- **Defensive programming:** checking every return value, validating input and bounding every buffer.
- **Testing:** designing test cases for both normal and faulty use and comparing results with standard tools.
- **Documentation:** recording the design, algorithms and results in a technical report.
- **Teamwork:** dividing the work among three students and combining it into one project.

Concepts understood

Team members gained a clear understanding of what happens inside a debugger: how a program is started under control, why a breakpoint is a patched byte, why RIP is one past the breakpoint, how single-stepping and system-call stops work, and how the kernel and the debugger cooperate through `ptrace()` and `waitpid()`. These concepts connect directly to the theory taught in the operating-systems course.

Challenges faced and how they were solved

Challenge	Solution
The program reports a stop one byte after the breakpoint	Compare RIP with `breakpoint_addr + 1` and set RIP back before resuming
A breakpoint traps again if left in place	Restore the original byte, single-step, then re-install it

Challenge	Solution
`PTRACE_PEEKDATA` returns -1 for valid data	Clear `errno` first and test it afterwards
Breakpoint addresses changed between runs	Build the target with `-no-pie`
Telling system-call stops from other traps	Enable `PTRACE_O_TRACESYSGOOD` and test for `SIGTRAP | 0x80`
Pairing system-call entry with exit	Toggle the `syscall_entry` flag after each stop
Window freezing during a long wait	Accepted as a limitation; a worker thread is proposed as future work

Team contribution

Fill in the table below with the part of the work done by each team member.

Student	Roll number	Contribution
SHETTENI TEJA	2520030419	[describe your part of the work]
A CHARANTEJA	2520030418	[describe your part of the work]
ASHWITH REDDY	2520030475	[describe your part of the work]

Reflection

The project showed that a few system calls, used carefully, are enough to build a working debugger. It also showed how much of the apparent complexity of real tools comes from features such as symbols and multiple threads, rather than from the basic mechanism. Understanding the basic mechanism makes the advanced features much easier to approach.

CHAPTER 9 REFERENCES

9.1 Books

The following books were used for background on operating systems, Linux programming and the C language. A short note says what each was used for.

1. Silberschatz, A., Galvin, P. B., and Gagne, G., *Operating System Concepts*, Wiley.
2. Kerrisk, M., *The Linux Programming Interface*, No Starch Press, 2010.
3. Bryant, R. E. and O'Hallaron, D. R., *Computer Systems: A Programmer's Perspective*, Pearson.
4. Kernighan, B. W. and Ritchie, D. M., *The C Programming Language*, Prentice Hall.
5. Love, R., *Linux Kernel Development*, Addison-Wesley.

How each book was used

Book	Used for
Operating System Concepts	Processes, process states, signals, system calls and memory management (Chapter 2)
The Linux Programming Interface	`fork`, `exec`, `waitpid`, signals and the detailed behaviour of `ptrace` (Chapters 4 and 6)
Computer Systems: A Programmer's Perspective	x86-64 registers, stack frames, system-call convention (Chapters 6 and 7)
The C Programming Language	C syntax, pointers, structures and bit operations (Section 4.1)
Linux Kernel Development	How the kernel handles processes, signals and system calls (Sections 2.1 and 6.6)

Notes on citation

Edition numbers and publication years should be added to match the copies actually consulted. The entries above give authors, titles and publishers; add the edition and year from the title page of your copy.

Suggested further reading

1. Stevens, W. R. and Rago, S. A., *Advanced Programming in the UNIX Environment*, Addison-Wesley.
2. Love, R., *Linux System Programming*, O'Reilly Media.
3. Bovet, D. P. and Cesati, M., *Understanding the Linux Kernel*, O'Reilly Media.
4. Tanenbaum, A. S. and Bos, H., *Modern Operating Systems*, Pearson.

Reference format

Use one format throughout the reference list. In IEEE style a book is written as: author initials

and surname, *title*, edition, publisher, year. For example:

```
[1] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts,
    10th ed. Hoboken, NJ, USA: Wiley, 2018.
```

The example shows the layout only. Use the edition and year of the copy you consulted.

9.2 Research Papers

The papers below describe well-known systems that observe or instrument running programs. They are related to the techniques used in this project and were used for background reading.

1. Nethercote, N. and Seward, J., "Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation", *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2007.
2. Luk, C.-K., et al., "Pin: Building Customized Program Analysis Tools with Dynamic Instrumentation", *Proceedings of PLDI*, 2005.
3. Bala, V., Duesterwald, E., and Banerjia, S., "Dynamo: A Transparent Dynamic Optimization System", *Proceedings of PLDI*, 2000.
4. O'Callahan, R., et al., "Engineering Record and Replay for Deployability", *USENIX Annual Technical Conference*, 2017.
5. Bruening, D., "Efficient, Transparent, and Comprehensive Runtime Code Manipulation", Ph.D. Thesis, Massachusetts Institute of Technology, 2004.

Relevance to this project

Paper	Relevance
Valgrind (PLDI 2007)	Observes a program without changing its source; shows an alternative to ptrace-based control
Pin (PLDI 2005)	Dynamic instrumentation: inserting analysis code into a running program
Dynamo (PLDI 2000)	Runtime monitoring and modification of a program's machine code
rr (USENIX ATC 2017)	A practical debugger that records and replays executions using ptrace
Bruening thesis (MIT, 2004)	Detailed treatment of runtime code manipulation, a close relative of breakpoint patching

Comparison of approaches

The Mini Linux Debugger controls a program from outside, using the operating system's tracing facility and one-byte code patches. The systems in the papers go further: they rewrite or record large amounts of code and execution. Studying them places the project in context: it uses the simplest member of the same family of techniques, which makes it a good first step toward those more advanced tools.

Add here any further papers on debugging, tracing or process control that you studied during the project.

Short summary of each paper

Valgrind (Nethercote and Seward, 2007) describes a framework for building tools that analyse

programs while they run, by translating their machine code on the fly. Its best-known tool detects memory errors.

Pin (Luk et al., 2005) presents a system that lets programmers insert their own analysis code into a running program through a programming interface, without changing the source.

Dynamo (Bala, Duesterwald and Banerjia, 2000) describes a system that watches a running native program and optimises its hot code at run time, transparently to the program.

rr (O'Callahan et al., 2017) describes a debugger for Linux that records a program's execution so that it can be replayed exactly, as many times as needed, while using the ordinary operating-system tracing interface.

Bruening (2004) is a doctoral thesis on manipulating a program's code while it runs; it describes the design of the DynamoRIO system.

Connection with the Mini Linux Debugger

All five works observe or alter a running program without help from its source code. The Mini Linux Debugger does so in the simplest possible way: it patches one byte, and it uses the operating system's tracing support to stop and resume the program. The papers show where the same ideas lead when they are developed further.

9.3 Linux / GNU Documentation

The official manual pages and project documentation were the main technical sources for the exact behaviour of each system call and tool.

1. Linux manual pages: `ptrace(2)`, `waitpid(2)`, `fork(2)`, `execve(2)`, `kill(2)`, `signal(7)`, `syscalls(2)`.
2. GNU Project Debugger (GDB) documentation – https://www.sourceware.org/gdb/documentation/
3. GCC online documentation – https://gcc.gnu.org/onlinedocs/
4. GNU Make manual – https://www.gnu.org/software/make/manual/
5. Intel 64 and IA-32 Architectures Software Developer's Manual (description of the INT3 instruction).

What each source was used for

Source	Used for
`ptrace(2)`	All ptrace requests, `PTRACE_O_TRACESYSGOOD`, behaviour of single-step and system-call stops
`waitpid(2)`	Meaning of the status word and the `WIF*` macros
`fork(2)`, `execve(2)`	Process creation and program replacement
`kill(2)`, `signal(7)`	SIGKILL, SIGTRAP and signal semantics
`syscalls(2)`	System-call numbers and names
GDB documentation	Behaviour of a professional debugger, used for cross-checking results
GCC and Make manuals	Compiler options (`-Wall`, `-no-pie`, `-O0`) and Makefile rules
Intel manual	INT3 as a one-byte opcode (`0xCC`) and the trap flag used for single-stepping

Sections of ptrace(2) worth reading

Section of the manual page	Relevant to
Description of `PTRACE_TRACEME`	Launch module
`PTRACE_PEEKDATA` and `PTRACE_POKEDATA`	Memory read and breakpoint insertion
`PTRACE_GETREGS` and `PTRACE_SETREGS`	Registers and RIP correction
`PTRACE_CONT` and `PTRACE_SINGLESTEP`	Run control
`PTRACE_SYSCALL` and the description of syscall-enter and syscall-exit stops	System-call trace
`PTRACE_O_TRACESYSGOOD`	Telling system-call stops from other SIGTRAP stops
Signal-delivery-stop	How signals reach a traced process

Related kernel documentation

- The Linux kernel documentation at https://www.kernel.org/doc/ covers the Yama security module, which controls ptrace permissions.
- The x86-64 system-call table is kept in the kernel source file `arch/x86/entry/syscalls/syscall_64.tbl`.

These two sources are useful for extending the debugger: the table lists every call number for the name table, and the Yama documentation explains permission problems.

9.4 Online Resources

The following online resources were used for explanations, examples and the GTK+ interface.

1. Linux man-pages online: https://man7.org/linux/man-pages/
2. ptrace(2) manual page: https://man7.org/linux/man-pages/man2/ptrace.2.html
3. GTK 3 reference documentation: https://docs.gtk.org/gtk3/
4. Eli Bendersky, "How debuggers work" article series: https://eli.thegreenplace.net/2011/01/23/how-debuggers-work-part-1
5. Ubuntu documentation: https://help.ubuntu.com/

Use of the online resources

Resource	Used for
man7.org Linux man-pages	Searchable online copy of the manual pages listed in Section 9.3
ptrace(2) page	Detailed description of each request used in the project
GTK 3 documentation	Widgets and signals used in `gui.c`
"How debuggers work" series	An easy introduction to breakpoints, INT3 and stepping on Linux
Ubuntu documentation	Package installation and system setup

Note on access dates

For a formal report, add the date on which each web page was last accessed next to its address, for example "Accessed: 11 October 2026".

Additional online resources

1. Linux kernel documentation: https://www.kernel.org/doc/
2. Yama security module: https://www.kernel.org/doc/html/latest/admin-guide/LSM/Yama.html
3. GDB manual (online): https://sourceware.org/gdb/current/onlinedocs/gdb/

Citing web pages

A web page should be cited with its author or organisation, title, address and the date it was accessed. In IEEE style:

```
[2] M. Kerrisk, "ptrace(2) - Linux manual page," man7.org.
    https://man7.org/linux/man-pages/man2/ptrace.2.html
    (accessed Oct. 11, 2026).
```

Evaluating online sources

- Prefer official documentation (manual pages, project sites) over forums.
- Check that a page matches the version of Linux and GTK that you use.

- Cross-check surprising statements against a second source or by running a small experiment, as this project does with `objdump`.